

CSI 2110
Data Structures and Algorithms

Lecture 1: Analysis of Algorithms

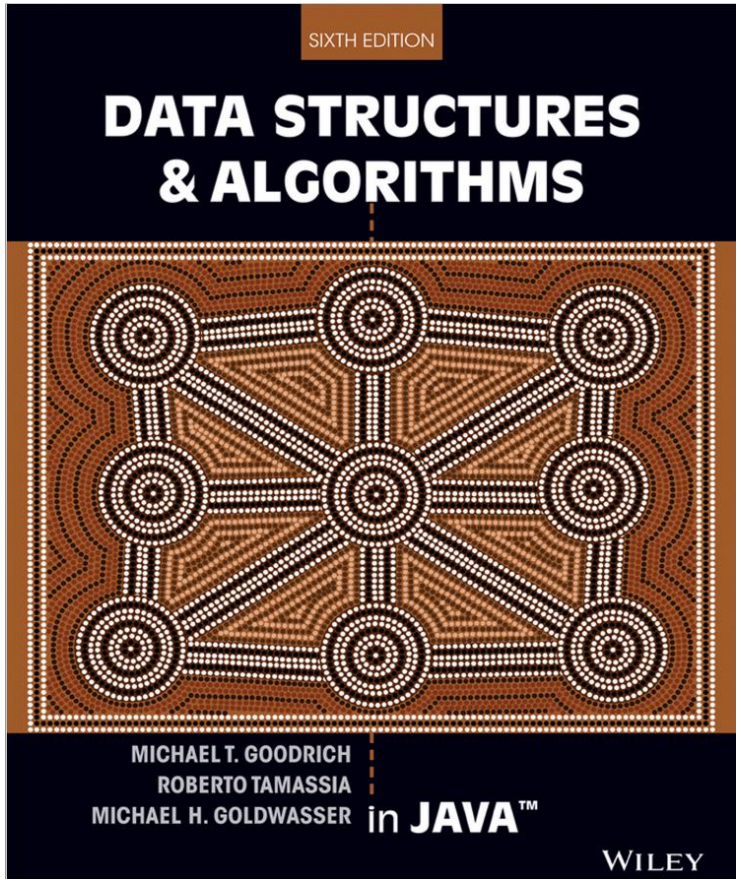
Bo Sun

Fall 2026

A note on the slides

Adapted from the slides that accompany this book.

Part of the slides is based on previous slides from Prof. Lucia Moura for CSI 2110.



Data Structures and Algorithms in Java
6th edition
Michael Goodrich, Roberto Tamassia, Michael Goldwasser
Wiley, 2014

Analysis of algorithms: roadmap

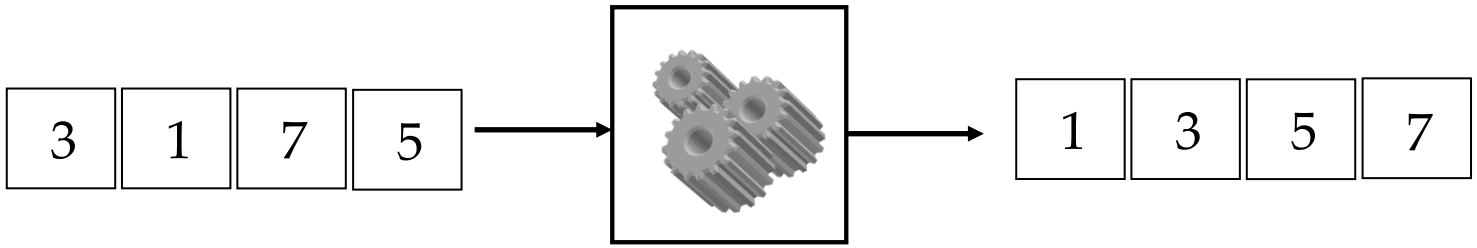
- What is an algorithm?

- Evaluating running time of an algorithm
  - experimental study
  - operations counting

- Big-*O*
  - Big-*O* definition
  - properties of Big-*O*
  - asymptotic analysis

What is an algorithm?

- An algorithm is a step-by-step procedure for solving a problem in a finite amount of time

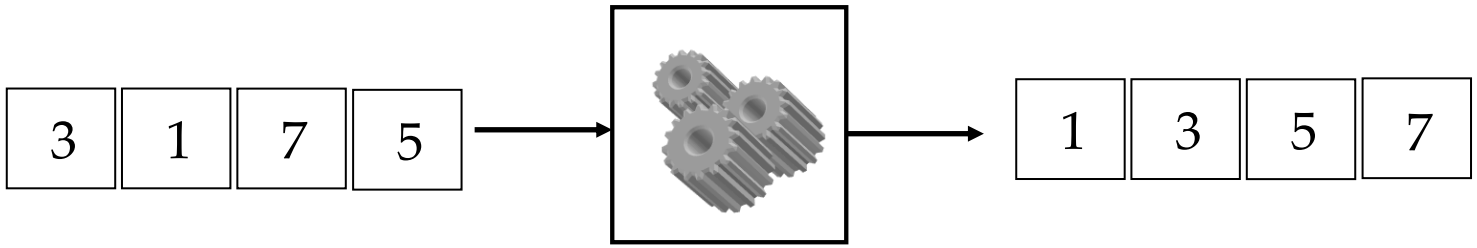


Example: a sorting algorithm

- Input: 4 integers $X = \{3, 1, 7, 5\}$
- Problem: arrange the input in an ascending order
- Algorithm: a well-defined step-by-step procedure to produce the output $Y = (1, 3, 5, 7)$

What do we care about when evaluating algorithms?

- An algorithm is a step-by-step procedure for solving a problem in a finite amount of time



Correctness

Is this algorithm doing what it is supposed to do?

Efficiency

How much time and space do we need to run this algorithm?

Implementation

Can we use less time and/or space? (data structures matter!!)

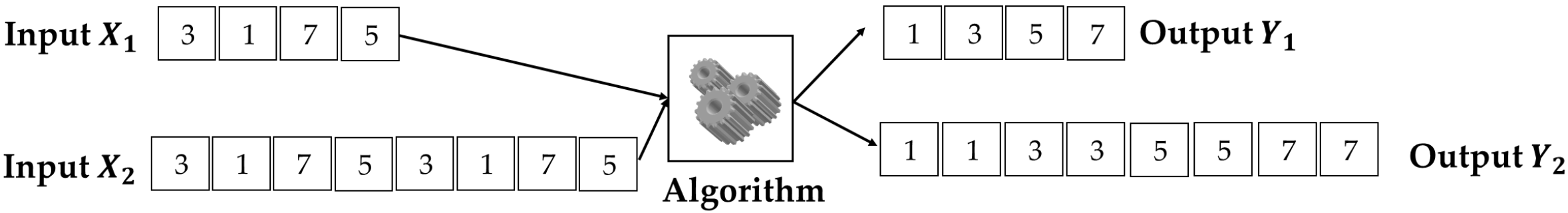
Efficiency of an algorithm

- Running time (RT): how much time does an algorithm take to execute?

- Memory: how much memory does an algorithm require?

What determines the running time (RT)?

- RT of an algorithm typically grows with the input size
  - Input size n: a measure of the amount of input data
    For an array/list, n is typically the number of elements



- RT also depends on input data: different inputs of the same size can have different RTs

Example: RT depends on the input data

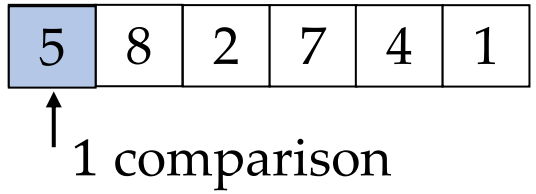
Input: an array A with n elements and a target x

Problem: search for the target x in array A

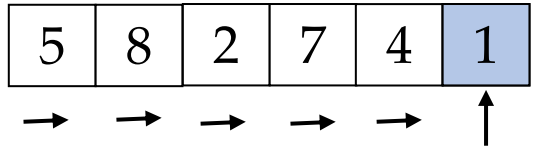
```
public static int search(int[] A, int x) {
    for (int i = 0; i < A.length; i++) {
        if (A[i] == x)                    ▷ if x is founded,
            return i;                     return its index
    }
    return -1;                            ▷ if x is not
}                                         founded, return −1
```

$A = [5, 8, 2, 7, 4, 1]$ $(n = A.\text{length} = 6)$

Case 1: $x = 5$



Case 2: $x = 1$



6 comparisons

Same input size $n = 6$, but different input data can result in different running times.

Analysis of algorithms: roadmap

- What is an algorithm?
- Evaluating running time of an algorithm
  - experimental study
  - operations counting
- Big-*O*
  - Big-*O* definition
  - properties of Big-*O*
  - asymptotic analysis

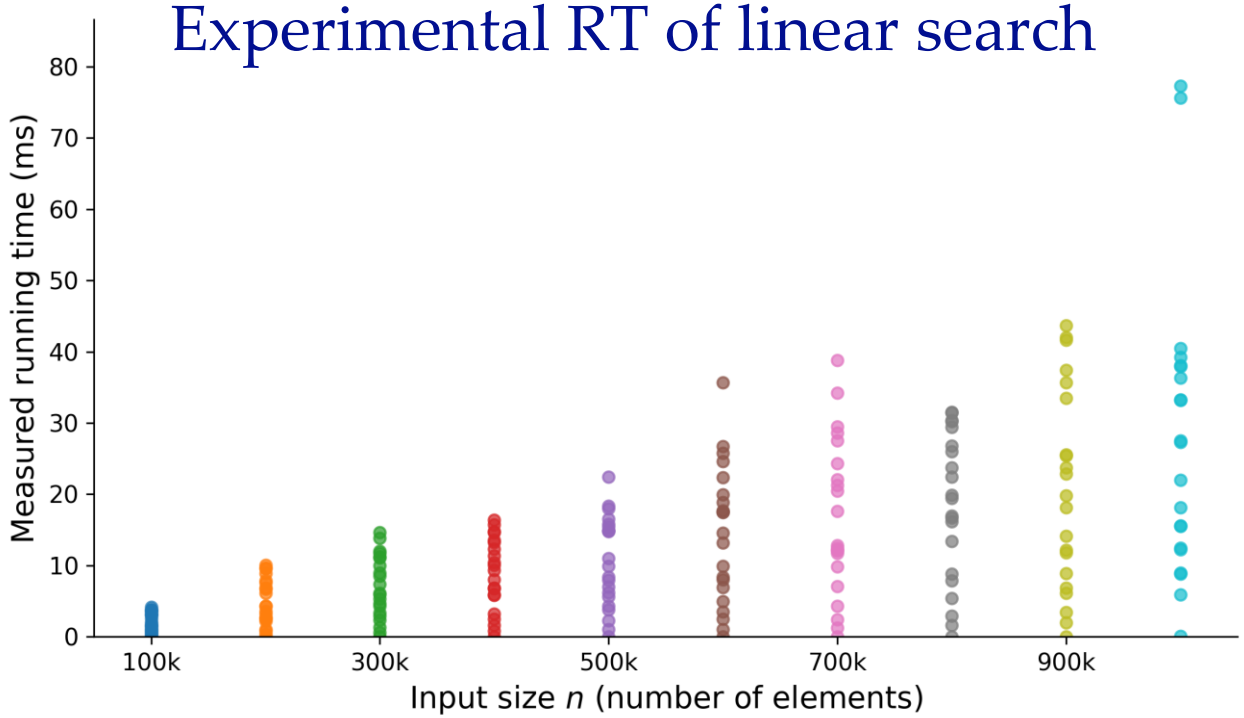
Experimental study of RT

- write a program that implements the algorithm
- run the program with inputs of varying size and data composition
- plot the results

```
1  long startTime = System.currentTimeMillis();
2  /* (run the algorithm) */
3  long endTime = System.currentTimeMillis();
4  long elapsed = endTime − startTime;
```

Java code to measure RT

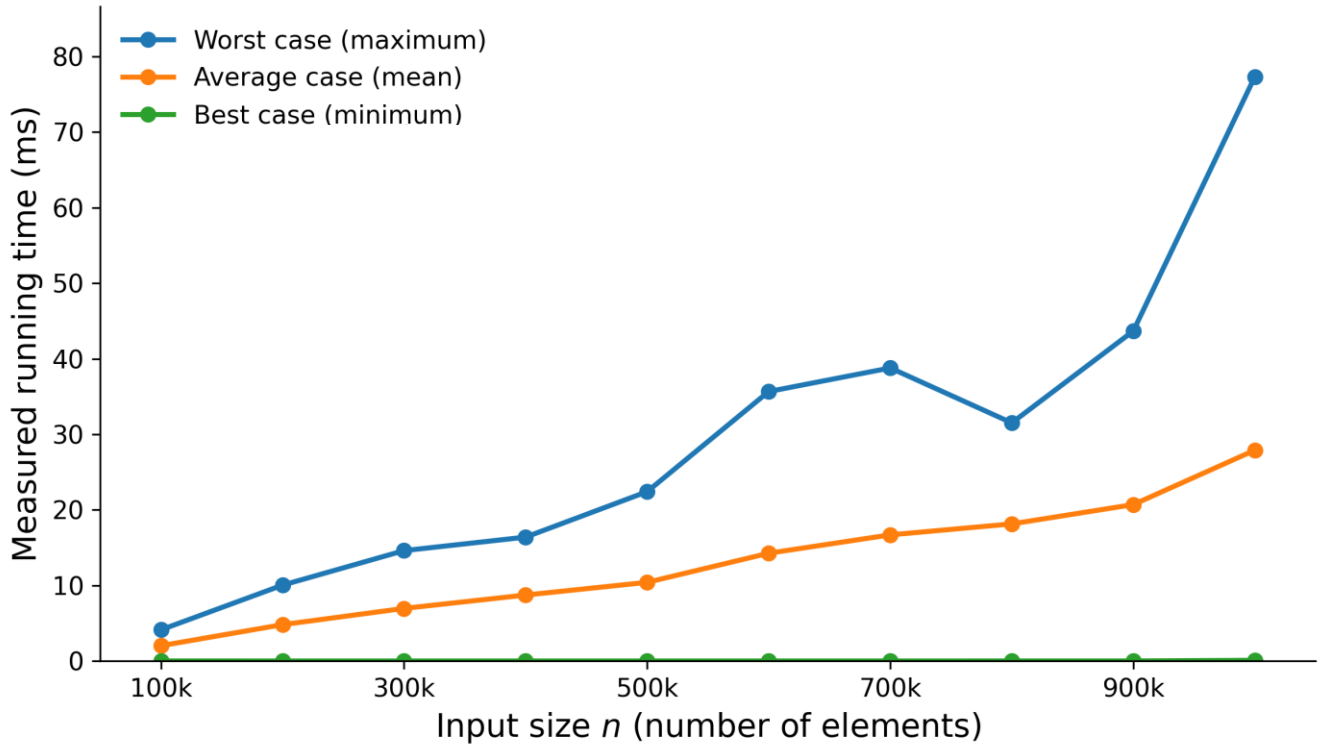
Experimental RT of linear search



RT of an algorithm

- Average case time is often difficult to analyze

- We mainly focus on worst-case running time
  - easier to analyze
  - crucial to applications such as games, finance and robotics



RT also depends on hardware and software!



RT: 10 days



RT: 10 minutes



RT: 10 seconds

- Measured RT are not machine or programming language independent!
- We need an analytic way of measuring RT independent of hardware and software environments (e.g., CPU speed, compiler, implementation)

Analysis of algorithms: roadmap

- What is an algorithm?
- Evaluating running time of an algorithm
  - experimental study
  - operations counting
- Big-*O*
  - Big-*O* definition
  - properties of Big-*O*
  - asymptotic analysis

Operations counting

- uses pseudocode to describe an algorithm, which is independent of implementations
- selects a theoretical computer model and identify key operations
- measures the efficiency by how many operations an algorithm needs, as a function of the input size

Pseudocode

- Pseudocode is a high-level description of an algorithm
  - an intermediate language between English and a programming language (such as Java or Python)
  - hiding program implementation details

Example: an algorithm that finds the maximum element of an array

```
Algorithm arrayMax(data)
Input: An integer array data
Output: The maximum element in data
n ← length(data)
currentMax ← data[0]
For i ← 1 to n − 1 do
    If data[i] > currentMax then
        currentMax ← data[i]
return currentMax
```

Pseudocode conventions

- Method declaration:

  Algorithm/Procedure *method (arg [, arg…])*
- Expressions

  ← Assignment = Equality testing
- Indentation indicates block structure
- Control flow

  for … do … if … then … [else …]

  repeat … until … while … do …

From pseudocode to code

```
Algorithm arrayMax(data)
Input: An integer array data
Output: The maximum element in data
n ← length(data)
currentMax ← data[0]
For i ← 1 to n − 1 do
    If data[i] > currentMaxthen
        currentMax ← data[i]
```

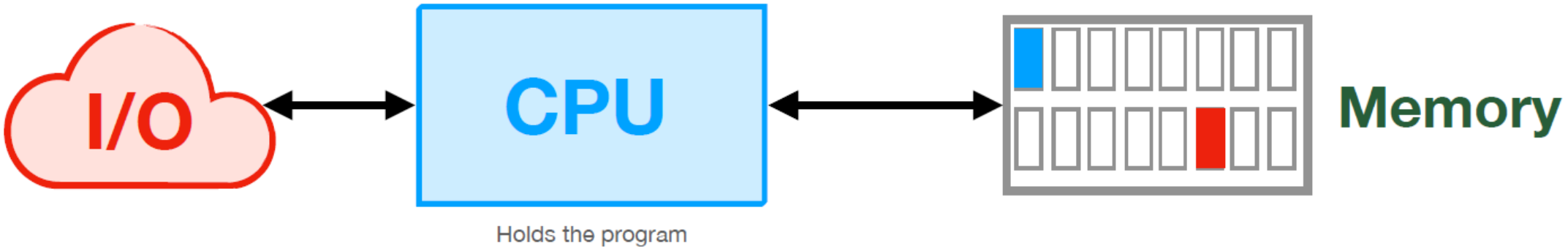
Python code:

```
def array_max(data)
        current_max= data[0]
        for val in data:
            if val > current_max
                current_max = val
        return current_max
```

Java code:

```
1  /** Returns the maximum value of a nonempty array of numbers. */
2  public static double arrayMax(double[ ] data) {
3    int n = data.length;
4    double currentMax = data[0];          // assume first entry is biggest (for now)
5    for (int j=1; j < n; j++)             // consider all other entries
6      if (data[j] > currentMax)           // if data[j] is biggest thus far...
7        currentMax = data[j];             // record it as the current max
8    return currentMax;
9  }
```

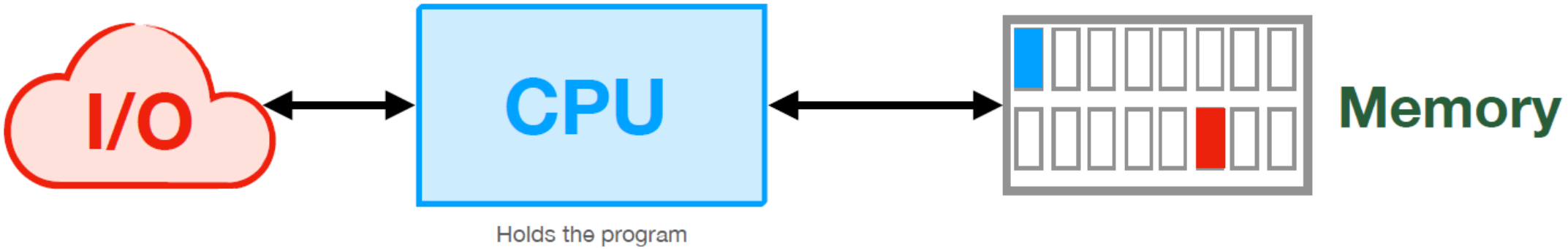
Model of Computation: Random Access Machine (RAM)



Components

- Input/output devices
- CPU: computation unit
- Memory: potentially unbounded number of cells, each of which can hold a number or character

RAM: Key Assumptions



- Memory access takes constant time
- Primitive operations take constant time
  - Assigning a value to a variable
  - Arithmetic operations (+, − , ×, /)
  - Comparing two variables
  - Returning a value from a method

All models are wrong, some are useful.

by George Box

Counting Primitive Operations

- Based on pseudocode of an algorithm, its maximum number of primitive operations is a function of the input size

```
Algorithm arrayMax(data)
Input: An integer array data
Output: The maximum element in data
0  n ← length(data)
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMax then
4           currentMax ← data[i]
5  return currentMax
```

Step 0: 2 ops

Step 1: 2 ops

Step 2: 2n ops

Step 3: 2(n-1) ops

Step 4: 0 to 2(n-1) ops

Step 5: 1 ops

Assignments and comparisons

Data

5	13	4	7	6	2	3	8	1	2

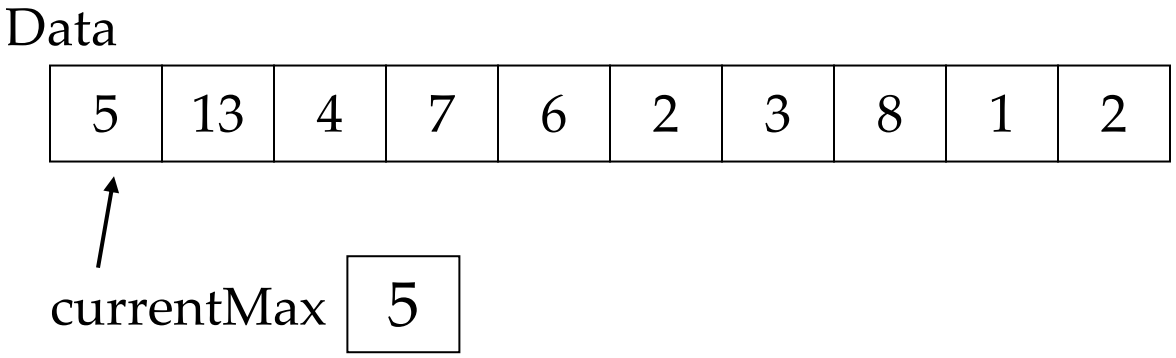
currentMax 

Assignments?

Comparisons?

```
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMaxthen
4           currentMax ← data[i]
5  Return currentMax
```

Assignments and comparisons



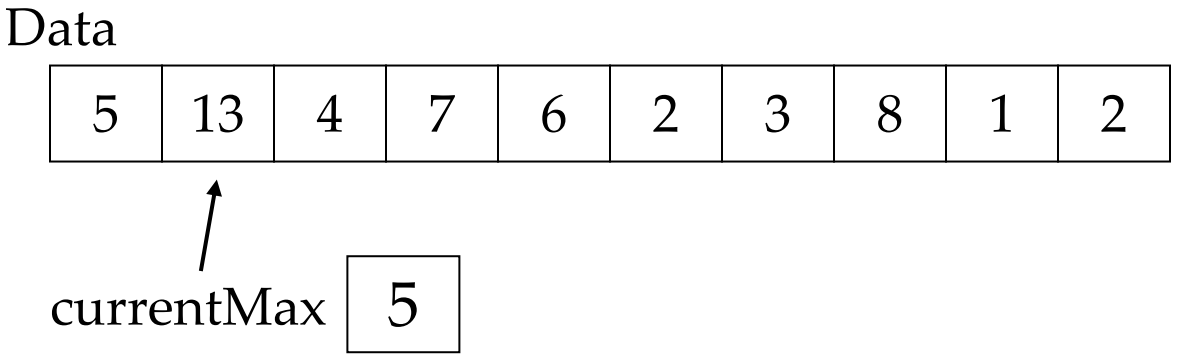
Assignments?



Comparisons?

```
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMax then
4           currentMax ← data[i]
5  Return currentMax
```

Assignments and comparisons

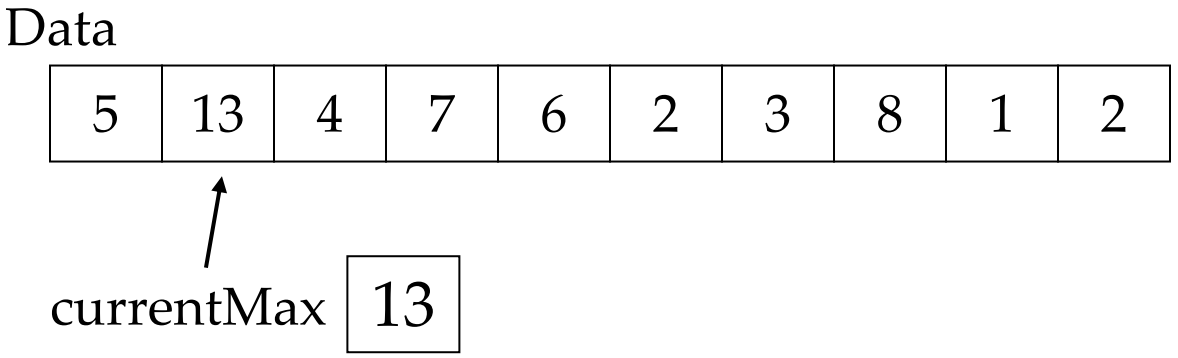


Assignments?

Comparisons?

```
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMax then
4        currentMax ← data[i]
5  Return currentMax
```

Assignments and comparisons

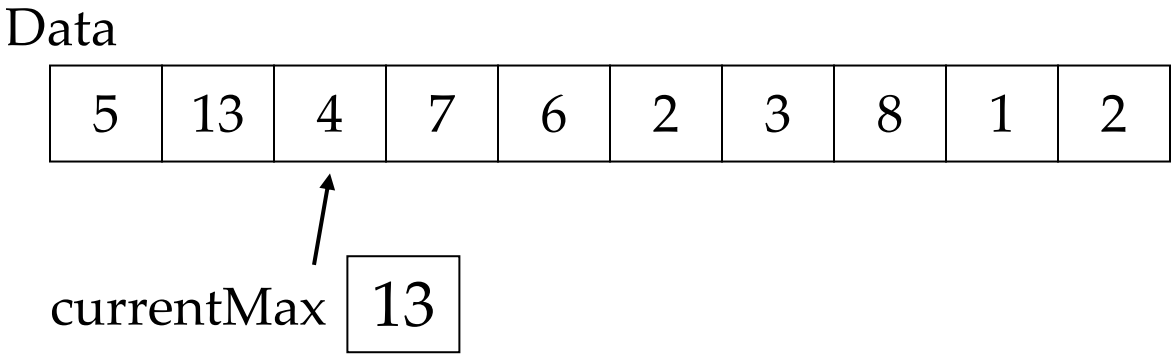


Assignments?

Comparisons?

```
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMax then
4           currentMax ← data[i]
5  Return currentMax
```

Assignments and comparisons

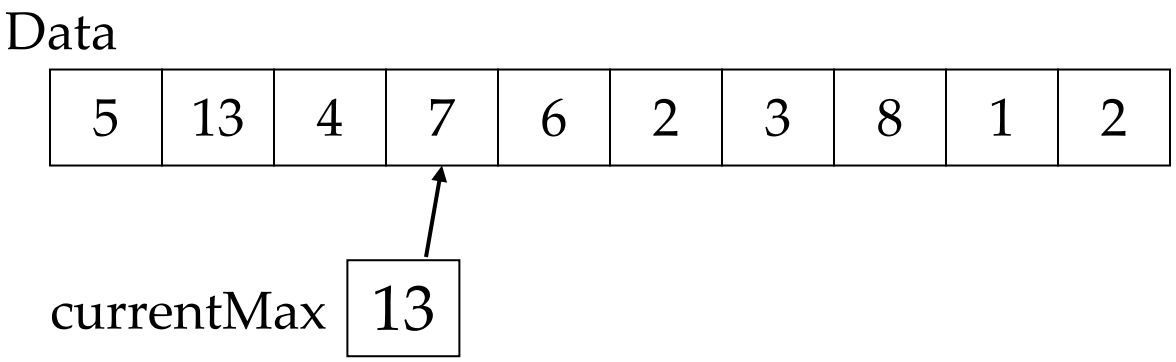


Assignments?

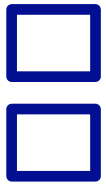
Comparisons?

```
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMax then
4        currentMax ← data[i]
5  Return currentMax
```

Assignments and comparisons



Assignments?

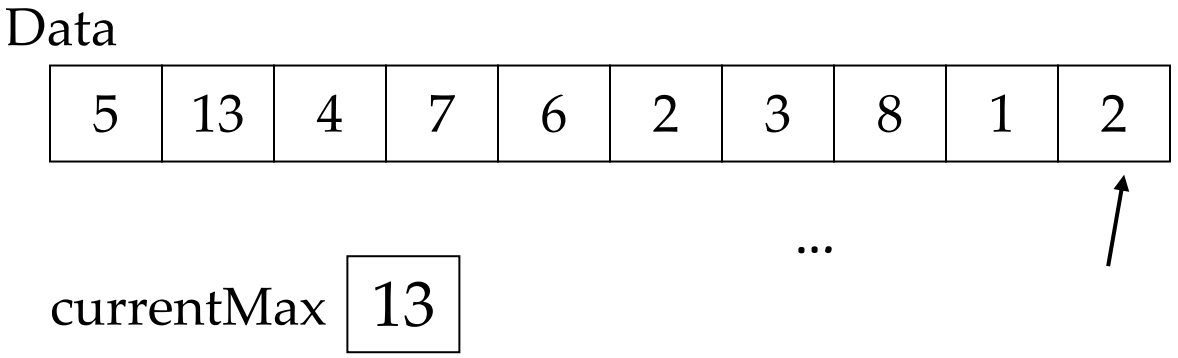


Comparisons?

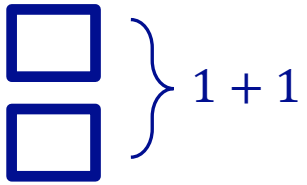


```
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMax then
4        currentMax ← data[i]
5  Return currentMax
```

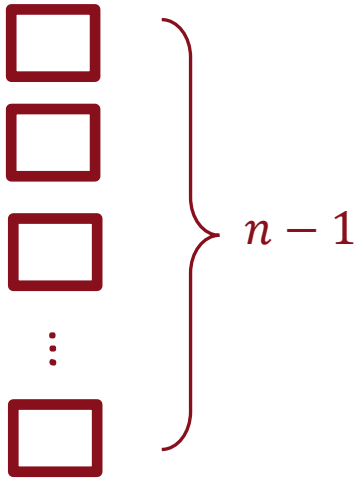
Assignments and comparisons



Assignments?



Comparisons?




```
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMax then
4        currentMax ← data[i]
5  Return currentMax
```

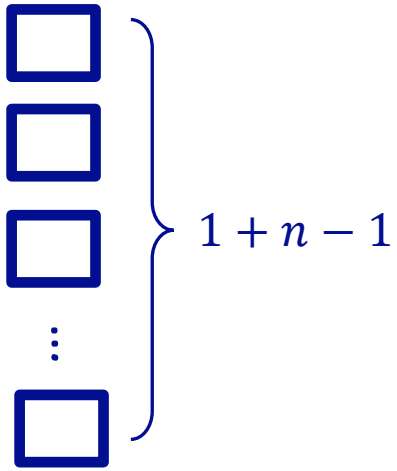
Assignments and comparisons: Worst Case

Data

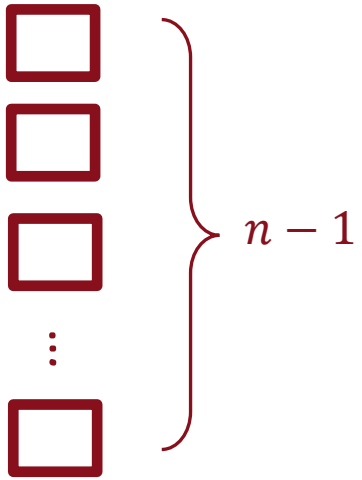
5	7	8	10	11	12	14	16	17	20

currentMax 

Assignments?



Comparisons?




```
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMax then
4           currentMax ← data[i]
5  Return currentMax
```

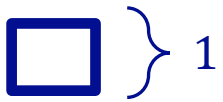
Assignments and comparisons: Best Case

Data

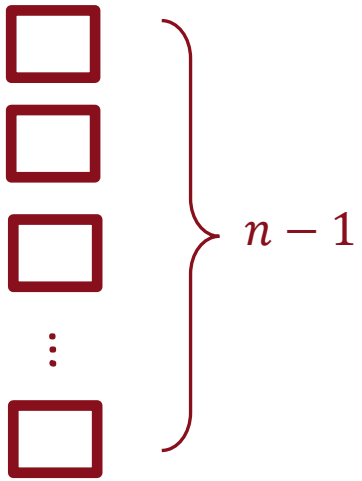
15	1	12	3	11	12	14	4	2	2

currentMax 

Assignments?



Comparisons?




```
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMax then
4           currentMax ← data[i]
5  Return currentMax
```

Counting primitive operations

```
Algorithm arrayMax(data)
Input: An integer array data
Output: The maximum element in data
0  n ← length(data)
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMaxthen
4           currentMax ← data[i]
5  Return currentMax
```

Step 0: 2 ops

Step 1: 2 ops

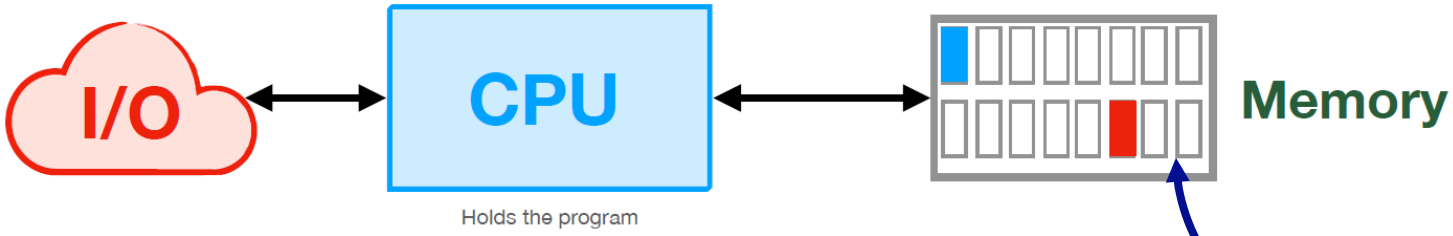
Step 2: 2n ops

Step 3: 2(n-1) ops

Step 4: 0 to 2(n-1) ops

Step 5: 1 ops

- Number of primitive operations of arrayMax
  - Best case: the first element is the maximum $4n + 3$
  - Worst case: elements in ascending order $6n + 1$



What operations your computers can do in a single operation with unit cost

Summary

Pseudocode;

Random access machine;

Example: arrayMax

Model of describing algorithms

Counting primitive operations: worst case and best case

```
Algorithm arrayMax(data)
Input: An integer array data
Output: The maximum element in data
 n ← length(data)
 currentMax ← data[0]
For i ← 1 to n − 1 do
   If data[i] > currentMaxthen
        currentMax ← data[i]
```

5	7	8	10	11	12	14	16	17	20

15	1	12	3	11	12	14	4	2	2

Estimating RT using operations counting

- Let $T(n)$ be RT of arrayMax.

$$a\,(4n + 3) \leq T(n) \leq b(6n + 1)$$

Time taken by the fastest primitive operation

Time taken by the slowest primitive operation

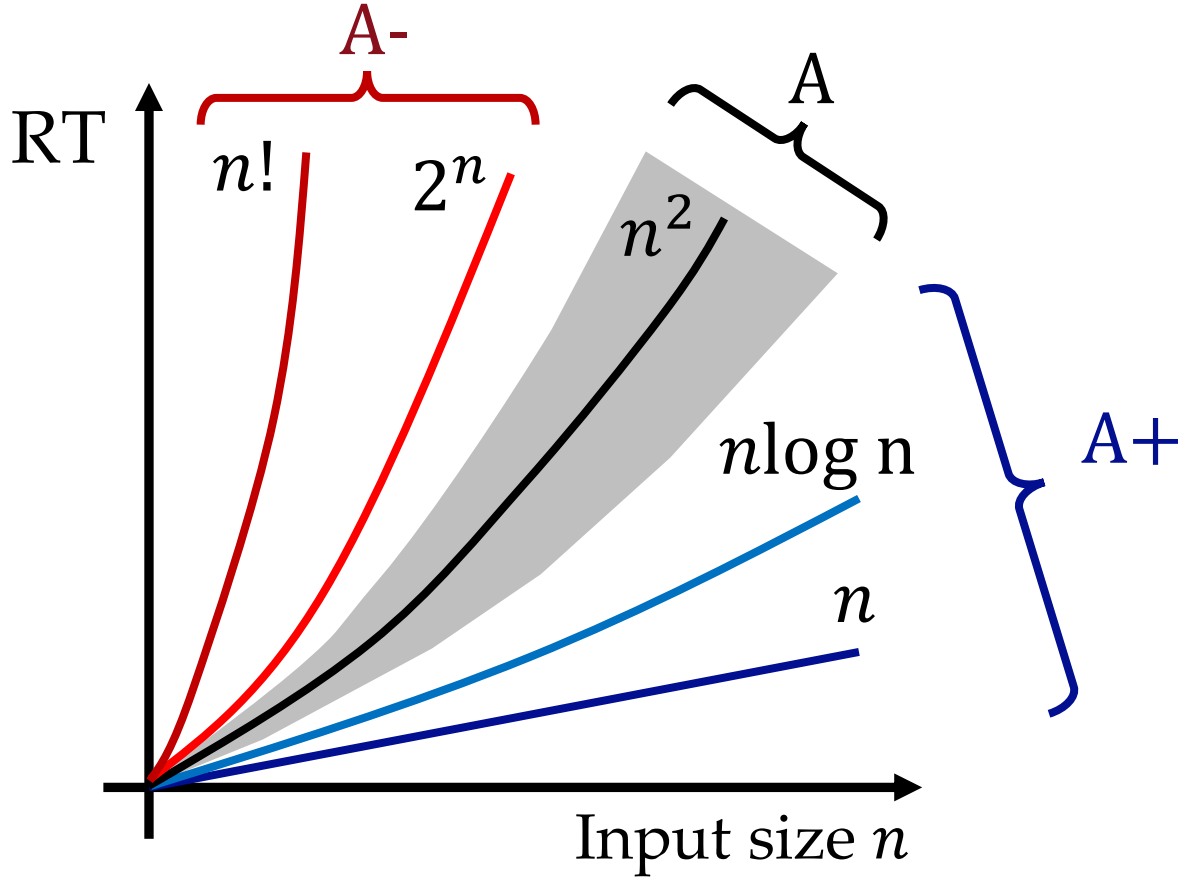
- The RT $T(n)$ of arrayMax grows linearly
  - changing hardware/software environment only affects $T(n)$ by a constant factor, but does not alter the growth rate
  - the linear growth rate of $T(n)$ is an intrinsic property of arrayMax

Analysis of algorithms: roadmap

- What is an algorithm?
- Evaluating running time of an algorithm
  - experimental study
  - operations counting
- Big-*O*
  - Big-*O* definition
  - properties of Big-*O*
  - asymptotic analysis

Asymptotic behavior

- The asymptotic behavior of RT $T(n)$ is the long-term trend of the growth rate as n becomes very large
  - in most cases, we only care about asymptotic behavior



Your grade

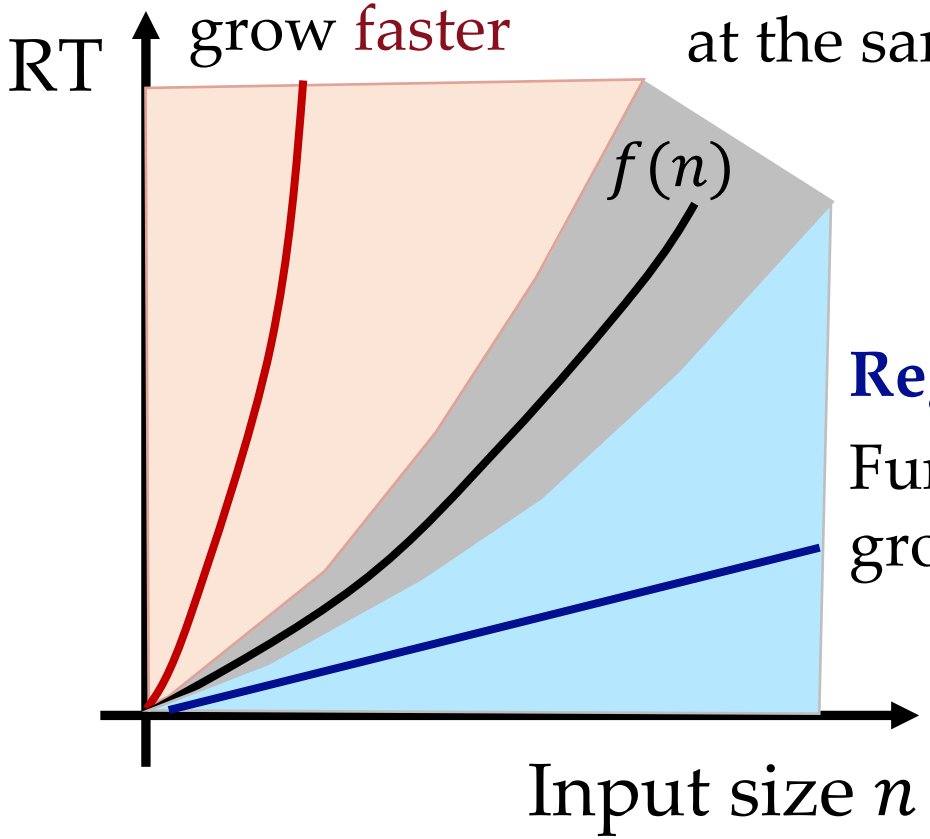
> 90%	→	A+
85 – 89%	→	A
80 – 84%	→	A-

A formal notation?

Asymptotic notations

Region 3:
Functions that grow faster

Region 1:
Functions that grow at the same rate



Region 2:
Functions that grow slower

Region 1 + Region 2 (“≤”)

- $O(f(n))$ – read as “big O of $f(n)$”
- $\Omega(f(n))$ – read as “big Ω of $f(n)$”
- $\Theta(f(n))$ – read as “Theta of $f(n)$”

Region 1 + Region 3 (“≥”)

Region 1 (“=”)

$$\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$$

Why asymptotic behavior matters?

Increase the input size	Increases slightly	doubles	Slightly more than doubles	Quadruples
n	$\log n$	n	$n \log n$	n^2
10	0.000003	0.00001	0.000033	0.0001
100	0.000007	0.0001	0.000664	0.1
1,000	0.00001	0.001	0.01	1
10,000	0.000013	0.01	0.1329	1.7 min
100,000	0.000017	0.1	1.661	2.78 hr
1,000,000	0.00002	1	19.9	11.6 day
1,000,000,000	0.00003	16.7 min	18.3 hr	318 century

Assume that the algorithm to solve the problem takes $f(n)$ microseconds

Some common functions

Polynomials: n^k (k is a constant, e.g., n, n^2, $\sqrt{n}$)

Logarithms: $\log n$

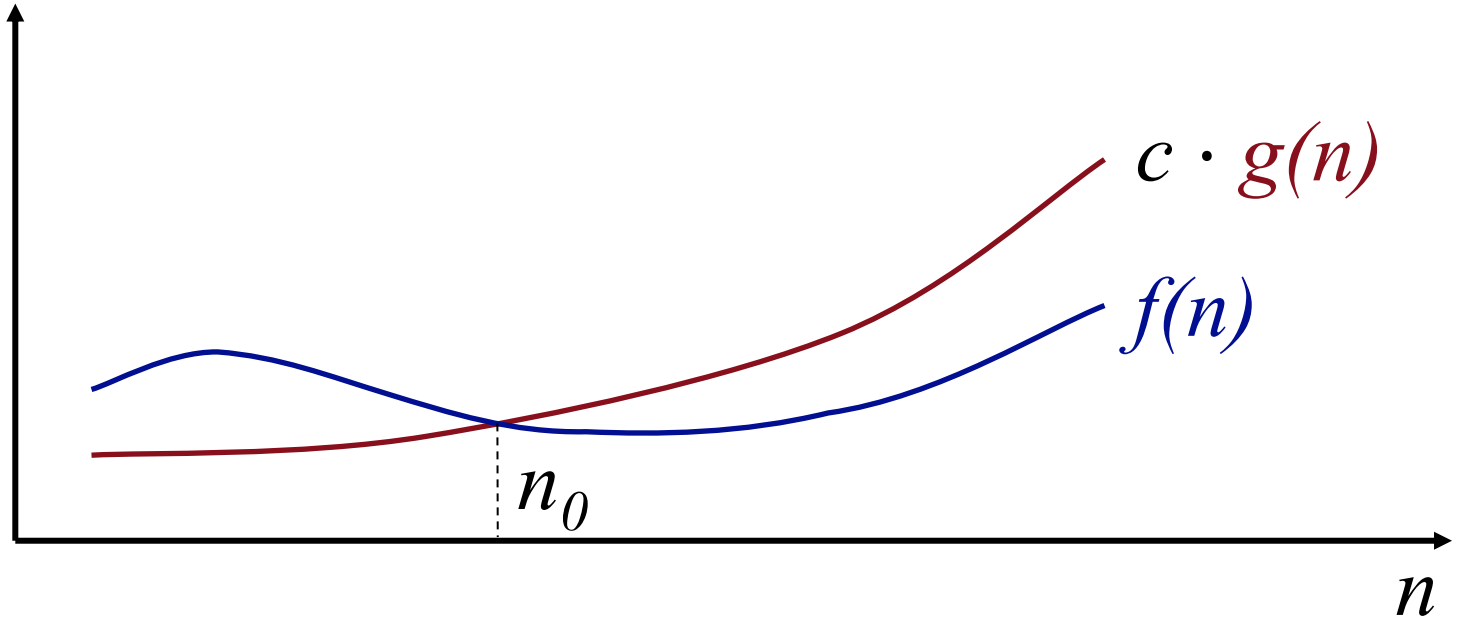
Exponents: a^n (a is a constant, e.g., 2^n, e^n)

Combinations of above (e.g., $n \log n$, $\log(\log n)$)

Big-*O*: Definition

- A function $f(n)$ belongs to $O(g(n))$ if and only if there exists constants $c > 0$ and $n_0 \in \mathbb{N}$ such that,

$$f(n) \leq c\ g(n) \quad \text{for all } n \geq n_0$$



Big-*O* Proof: Example 1

Prove that $f(n) = 25n + 2110$ is $O(n)$

$c = 2135$

$n_0 = 1$

$25n \leq c \cdot n$ when $c = 25$ for all values of $n \geq 1$

$2110 \leq c \cdot n$ when $c = 2110$ for all values of $n \geq 1$

$25n + 2110 \leq 25n + 2110n = 2135n$ all values of $n \geq 1$

$f(n) \in O(g(n))$ $\Leftrightarrow$ there exist c, n_0 such that $f(n) \leq c\ g(n)$ for all $n \geq n_0$

Big-*O* Proof: Example 2

Prove that $f(n) = 2n + 1$ is $O(n^2)$

$c = 2$

$n_0 = 2$

$2n \leq c \cdot n^2$ when $c = 1$ for all $n \geq 2$

$1 \leq c \cdot n^2$ when $c = 1$ for all $n \geq 1$

Therefore, $2n + 1 \leq 2n^2$ for all $n \geq 2$

$f(n) \in O(g(n))$ $\Leftrightarrow$ there exist c, n_0 such that $f(n) \leq c\ g(n)$ for all $n \geq n_0$

Non-uniqueness of c and n_0

$c = 3$

$n_0 = 1$

Prove that $f(n) = 2n + 1$ is $O(n^2)$

$2n \leq c \cdot n^2$ when $c = 2$ for all $n \geq 1$

$1 \leq c \cdot n^2$ when $c = 1$ for all $n \geq 1$

Therefore, $2n + 1 \leq 3n^2$ for all $n \geq 1$

$f(n) \in O(g(n))$ $\Leftrightarrow$ there exist c, n_0 such that $f(n) \leq c\ g(n)$ for all $n \geq n_0$

Big-*O* Proof by Definition

- **Step 1**: find c and n_0 for each term of $f(n)$
- **Step 2:** add up all your c, take the maximum of your n_0
- **Step 3**: add up all your inequalities to get the one inequality
- **Step 4**: write down what is c and n_0

$f(n) \in O(g(n)) \iff$ there exist c, n_0 such that $f(n) \leq c\ g(n)$ for all $n \geq n_0$

Big-*O* Proof: More Examples

Prove that $f(n) = 7n - 2$ is $O(n)$

$7n - 2 \leq 7n$ for all $n \geq 1$ $(c = 7, n_0 = 1)$

Prove that $f(n) = 3n^3 + 20n^2 + 5$ is $O(n^3)$

$3n^3 + 20n^2 + 5 \leq 28n^3$ for all $n \geq 1$ $(c = 28, n_0 = 1)$

Prove that $f(n) = 3\log n + 5$ is $O(\log n)$

$3\log n + 5 \leq 8\log n$ for all $n \geq 2$ $(c = 8, n_0 = 2)$

Big-*O* Proof: Polynomials

Prove that for any $d > 0$, $f(n) = \sum_{j=0}^{d} a_j n^j$ is

Proof: It can be shown that

$$f(n) = \sum_{j=0}^{d} a_j n^j \leq \sum_{j=0}^{d} |a_j| n^d, \text{ for all } n \geq 1$$

Therefore, $f(n) = O(n^d)$ with c $= \sum_{j=0}^{d} |a_j|$ and $n_0 = 1$

Big-O conventions

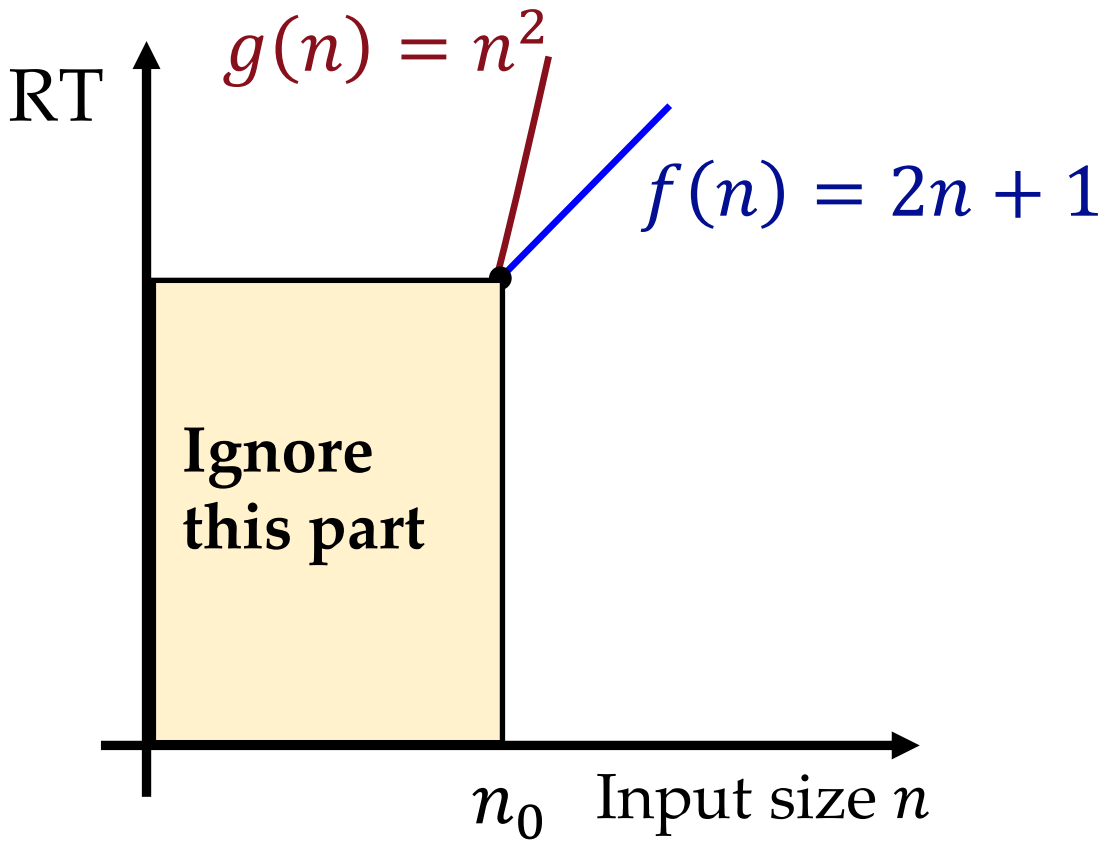
- If $f(n)$ is a polynomial of degree d, then $f(n)$ is $O(n^d)$, i.e.,
  - Drop lower order terms
  - Drop constant factors

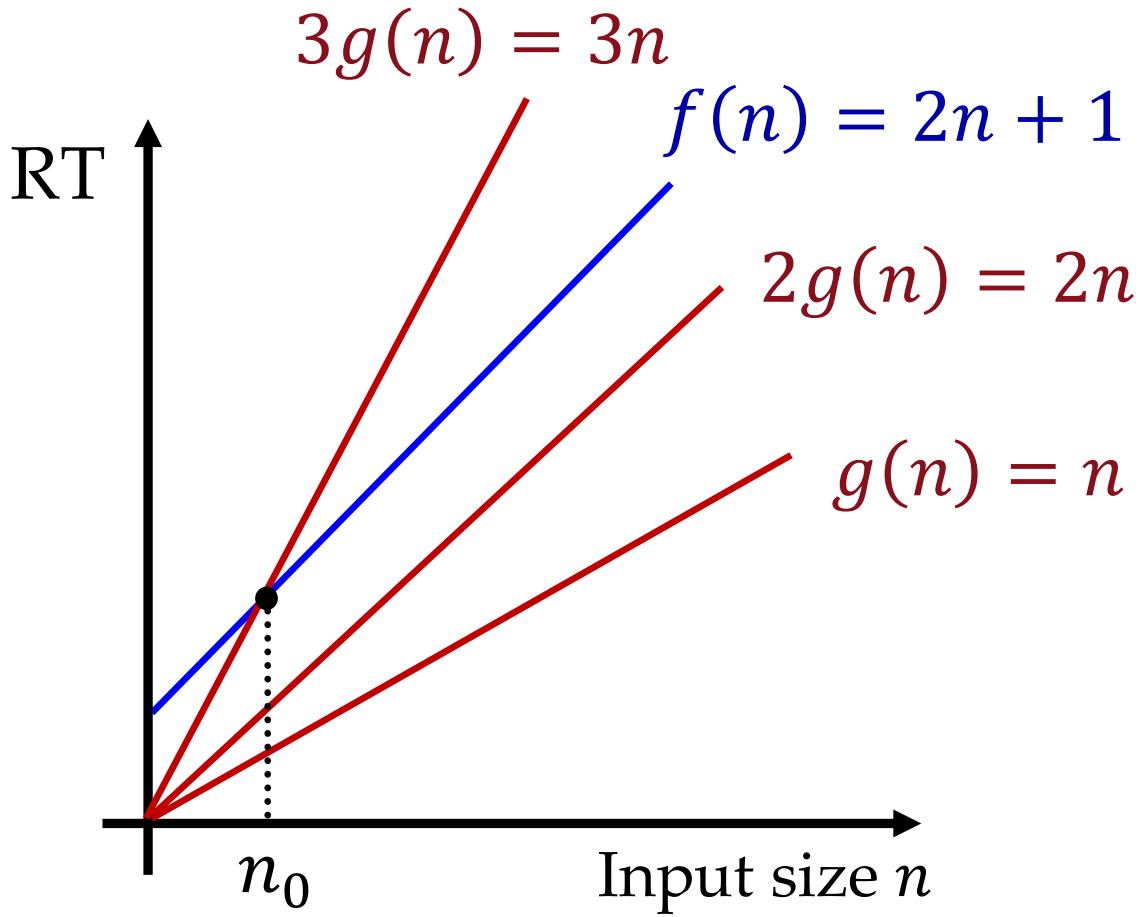
  Examples: $7n-3$ is $O(n)$; $8n^2\log n + 5n^2 + n$ is $O(n^2\log n)$;
  $12n^3 + 5000n^2 + 2n^4$ is $O(n^4)$
- Use the smallest possible class of functions
  - we say $2n$ is $O(n)$ instead of $2n$ is $O(n^2)$

  Both are correct, but the first is tighter
- Use the simplest expression of the class
  - we say $3n + 5$ is $O(n)$ instead of $3n + 5$ is $O(3n)$

Big-*O*: Why we need both n_0 and c?

$f(n) \in O(g(n)) \iff$ there exist c, n_0 such that $f(n) \leq c\ g(n)$ for all $n \geq n_0$





Big-O: Constant c is important!

Suppose this is the definition of Big-O (constant $c = 1$)

$f(n) \in O(g(n)) \iff$ there exist c, n_0 such that $f(n) \leq g(n)$ for all $n \geq n_0$

Big-O with c

$n \in O(n)$

$n + 1 \in O(n)$

$2n + 1 \in O(n)$

Linear increment

Big-O with $c = 1$

$n \in O(n)$

$n + 1 \notin O(n)$

$2n + 1 \notin O(n)$

$n + 1 > n$ holds for all $n \geq 1$

These would be different!

$O(n)$

$O(n + 0.01)$

$O(c\,n)$

Big-O would be a mess!

Some remarks on Big-O proof

- The values of c and n_0 are not unique
- You do NOT need to find the smallest possible c and n_0
- Both n_0 and c are important in the Big-O definition
- $O(g(n))$ is just an upper bound (not necessarily tight)
  - $2n + 1 \in O(n)$ and $2n + 1 \in O(n^2)$

Analysis of algorithms: roadmap

- What is an algorithm?
- Evaluating running time of an algorithm
  - experimental study
  - operations counting
- Big-*O*
  - Big-*O* definition
  - properties of Big-*O*
  - asymptotic analysis

Properties of Big-*O*

Prove that for any $f(n), g(n) > 0$, $f(n) + g(n)$ is $O(\max\{f(n), g(n)\})$

Since $f(n), g(n) > 0$, we have

$f(n) + g(n) \leq 2 \max\{f(n), g(n)\}$ all values of $n \geq 1$

Therefore, $f(n) + g(n)$ is $O(\max\{f(n), g(n)\})$ with $n_0 = 1$ and $c = 2$

Prove that for any $f(n), g(n) > 0$, $\max\{f(n), g(n)\}$ is $O(f(n) + g(n))$

Since $f(n), g(n) > 0$, we have

$\max\{f(n), g(n)\} \leq f(n) + g(n)$ all values of $n \geq 1$

Therefore, $\max\{f(n), g(n)\}$ is $O(f(n) + g(n))$ with $n_0 = 1$ and $c = 1$

Properties of Big-*O*

Analogy

- Reflexivity: $f(n)$ is $O(f(n))$

$$f \leq f$$

- Additivity: if $f(n), g(n) \in O(h(n))$, then $f(n) + g(n) \in O(h(n))$

$$f \leq h$$
$$g \leq h$$

- Multiplicative: if $f_1(n) \in O(f_2(n))$ and $g_1(n) \in O(g_2(n))$, and all functions are positive, then $f_1(n) \cdot g_1(n) \in O(f_2(n) \cdot g_2(n))$

$$f_1 \leq f_2$$
$$g_1 \leq g_2$$

- Transitivity: if $f(n) \in O(g(n))$ and $g(n) \in O(h(n))$, then $f(n) \in O(h(n))$

$$f \leq g \leq h$$

Functions you need to know

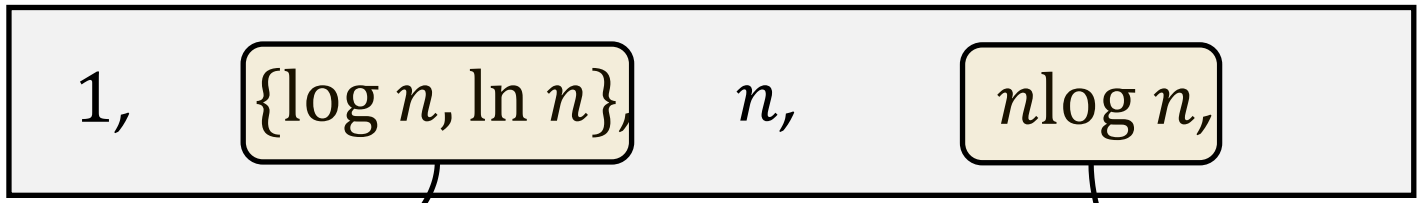
Constant	Logarithmic	Linear	Linearithmic
$1,$	$\{\log n, \ln n\},$	$n,$	$n\log n,$



Quadratic	Cubic	Exponential			Factorial
$n^2,$	$n^3,$	$2^n,$	$n \cdot 2^n,$	$e^n,$	$n!$



- Those functions are ordered in increasing order of growth, each is in Big-*O* of next one.



$$\log n = \frac{\ln n}{\ln 2}$$

$$n \in O(n)$$

$$1 \in O(\log n)$$

Then, $n \in O(n \log n)$

Property of logarithms

$$\log_b a = \frac{\log_x a}{\log_x b}$$

$$\log_b x^a = a \log_b x$$

$1,\quad \{\log n, \ln n\},\quad n,\quad n\log n,$

$n^2,\quad n^3,\quad 2^n,\quad n \cdot 2^n,\quad e^n,\quad n!$

Since $3\log n \leq n$, for all $n \geq 16$, we have

$n^3 = 2^{3\log n} \leq 2^n$, for all $n \geq 16$

Then $n^3 \in O(2^n)$, with $n_0 = 16$ and $c = 1$

Note $e^n \geq (2 \cdot 1.35)^n = 1.35^n \cdot 2^n$, and $n \in O(1.35^n)$, then $n \cdot 2^n \in O(e^n)$

$1, \quad \{\log n, \ln n\}, \quad n, \quad n\log n,$

$n^2, \quad n^3, \quad 2^n, \quad n \cdot 2^n, \quad e^n, \quad n!$

Factorial grows faster than exponential

Stirling's approximation (optional)

$$n! \approx \sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n$$

Summary for Big-*O*

- You need to develop an intuitive understanding of some common Big-*O* notations

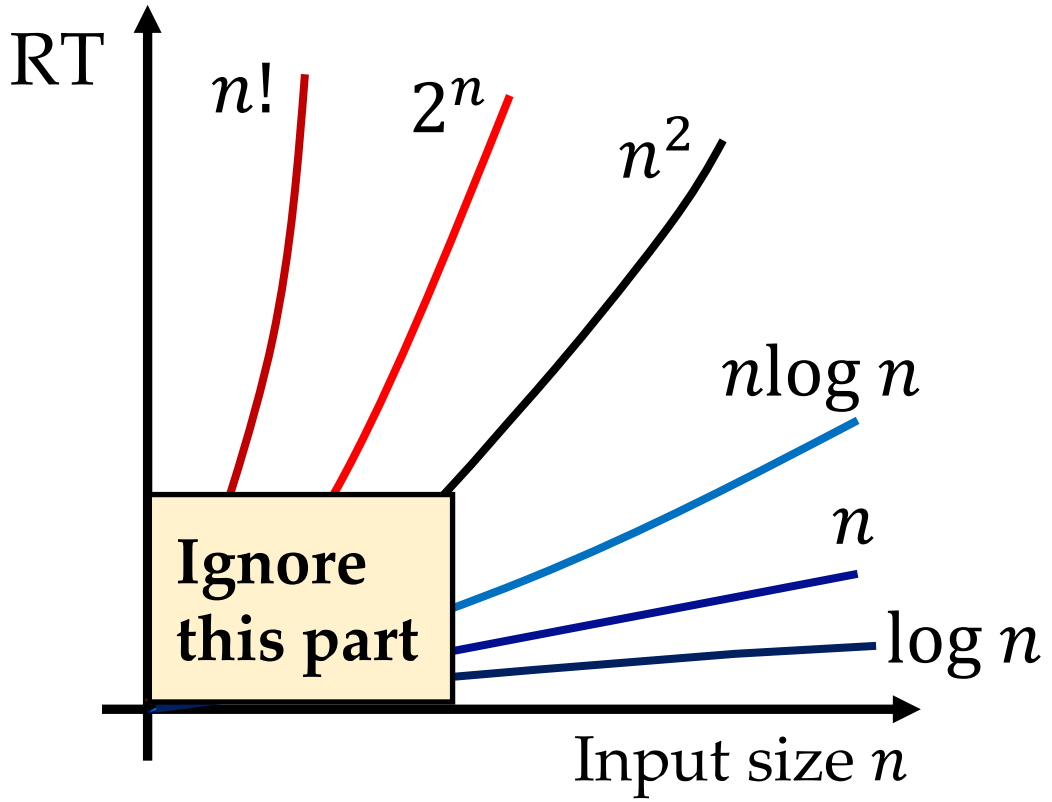
Some common functions

Polynomials: n^k (k is a constant, e.g., n, n^2, $\sqrt{n}$)

Logarithms: $\log n$

Exponents: a^n (a is a constant, e.g., 2^n, e^n)

Combinations of above (e.g., $n\log n$, $\log(\log n)$)



Analysis of algorithms: roadmap

- What is an algorithm?
- Evaluating running time of an algorithm
  - experimental study
  - operations counting
- Big-*O*
  - Big-*O* definition
  - properties of Big-*O*
  - asymptotic analysis

Asymptotic algorithm analysis

- The asymptotic analysis of an algorithm determines the running time in big-O notations

- To perform the asymptotic analysis
  - Find the worst-case number of operations as a function of the input size n
  - Express this function with big-O notation

  Examples: since arrayMax has worst-case $6n + 1$ primitive operations, we say arrayMax algorithm runs in $O(n)$ time

- Since lower order terms and constant factors are eventually dropped, we can disregard them when counting primitive operations

Asymptotic analysis: Example for arrayMax

```
Algorithm arrayMax(data)
Input: An integer array data
Output: The maximum element in data
0  n ← length(data)
1  currentMax ← data[0]
2  For i ← 1 to n − 1 do
3     If data[i] > currentMax then
4        currentMax ← data[i]
5  Return currentMax
```

Step 0: 2 ops — Step 0: $O(1)$

Step 1: 2 ops — Step 1: $O(1)$

Step 2: 2n ops — Step 2: $O(n)$

Step 3: 2(n-1) ops — Step 3: $O(n)$

Step 4: 0 to 2(n-1) ops — Step 4: $O(n)$

Step 5: 1 ops — Step 5: $O(1)$

The arrayMax algorithm runs in $O(n)$ time

Exercises for asymptotic analysis

Input array A is of size n

```
i ← 1;
While i < n do
    A[i] ← A[i] + 1;
    i ← i + 1;
```

$O(n)$

```
i ← 1;
While i < n do
    A[i] ← A[i] + 1;
    i ← i + 2;
```

$O(n)$

```
i ← 1;
While i < n do
    A[i] ← A[i] + 1;
    i ← i * 2;
```

$O(\log n)$

Exercises for asymptotic analysis

Input array A is of size n

```
For i ← 0 to 1000 do
    A[i] ← A[i] + 1;
```

$O(1)$

```
For i ← 0 to n − 1 do
    For j ← 0 to n − 1 do
        A[i] ← A[i] + A[j];
```

$O(n^2)$

```
For i ← 0 to n − 1 do
    For j ← 0 to i do
        A[i] ← A[i] + A[j];
```

$O(n^2)$

Arithmetic progression

$$f(n) = \sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \in O(n^2)$$

Proof 1 (Algebra):

$$f(n) = 1 + 2 + 3 + \cdots + (n-2) + (n-1) + n$$

$$f(n) = n + (n-1) + (n-2) + \cdots + 3 + 2 + 1$$

$$2f(n) = (n+1) + (n+1) + (n+1) + \cdots + (n+1)$$

(n copies)

$$\Rightarrow f(n) = \frac{n(n+1)}{2}$$

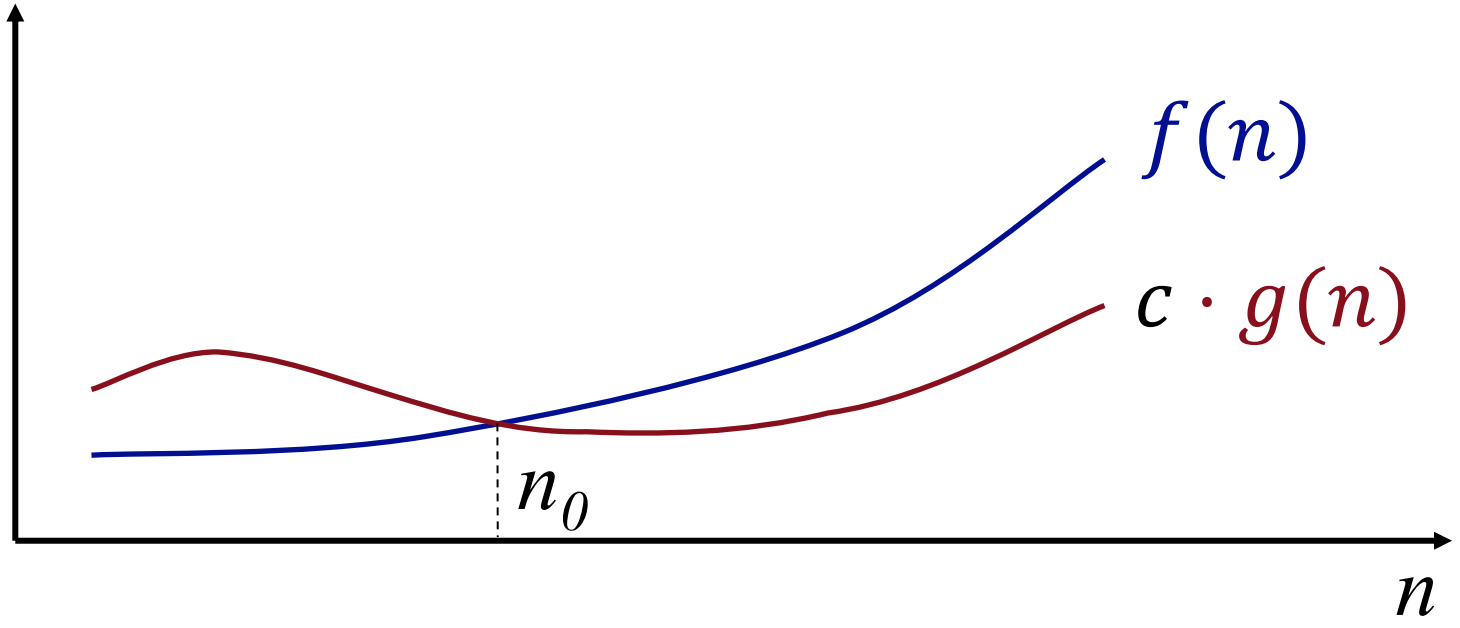
Analysis of algorithms: roadmap

- What is an algorithm?
- Evaluating running time of an algorithm
  - experimental study
  - operations counting
- Big-*O*
  - Big-*O* definition
  - properties of Big-*O*
  - asymptotic analysis
- Big-Ω and Big-Θ
- Comparing two algorithms
- Math reviews

Big-Ω: Definition

- A function $f(n)$ belongs to $\Omega(g(n))$ if and only if there exists constants $c > 0$ and $n_0 \in \mathbb{N}$ such that,

$$f(n) \geq c\ g(n) \quad \text{for all } n \geq n_0$$



Big-Ω Examples

Prove that $f(n) = 20n^2 + 4n$ is $\Omega(n^2)$

$20n^2 + 4n \in O(n^2)$

$20n^2 + 4n \geq 20\,n^2$ for all $n \geq 1$ $(c = 20, n_0 = 1)$

Prove that $f(n) = 20n^2 + 4n$ is $\Omega(\log n)$

$20n^2 + 4n \geq \log n$ for all $n \geq 1$ $(c = 1, n_0 = 1)$

Prove that $f(n) = n\log n + 204n$ is $\Omega(n\log n)$

$n\log n + 204n \in O(n\log n)$

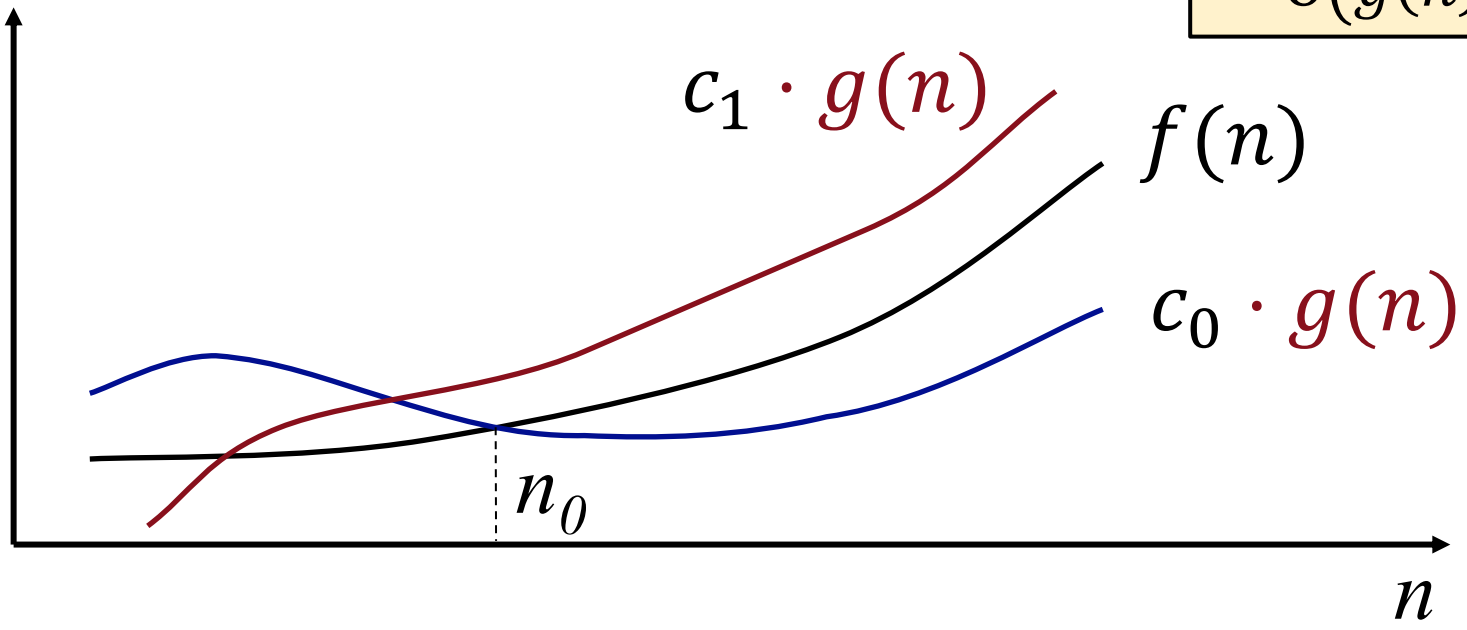
$n\log n + 204n \geq n\log n$ for all $n \geq 1$ $(c = 1, n_0 = 1)$

Big-Θ: Definition

- A function $f(n)$ belongs to $\Theta(g(n))$ if and only if there exists constants $c_0 > 0$, $c_1 > 0$ and $n_0 \in \mathbb{N}$ such that,

$$c_0 g(n) \leq f(n) \leq c_1\, g(n) \quad \text{for all } n \geq n_0$$

$$\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$$



Big-Θ Examples

Prove that $f(n) = 20n^2 + 4n$ is $\Theta(n^2)$

$20n^2 + 4n \geq 20\, n^2$ for all $n \geq 1$ $(c_0 = 20, n_0 = 1)$

$20n^2 + 4n \leq 24\, n^2$ for all $n \geq 1$ $(c_1 = 24, n_0 = 1)$

Prove that $f(n) = n\log n + 204n$ is $\Theta(n\log n)$

$n\log n + 204n \geq n\log n$ for all $n \geq 1$ $(c_0 = 1, n_0 = 1)$

$n\log n + 204n \leq 205n\log n$ for all $n \geq 1$ $(c_1 = 205, n_0 = 1)$

More examples

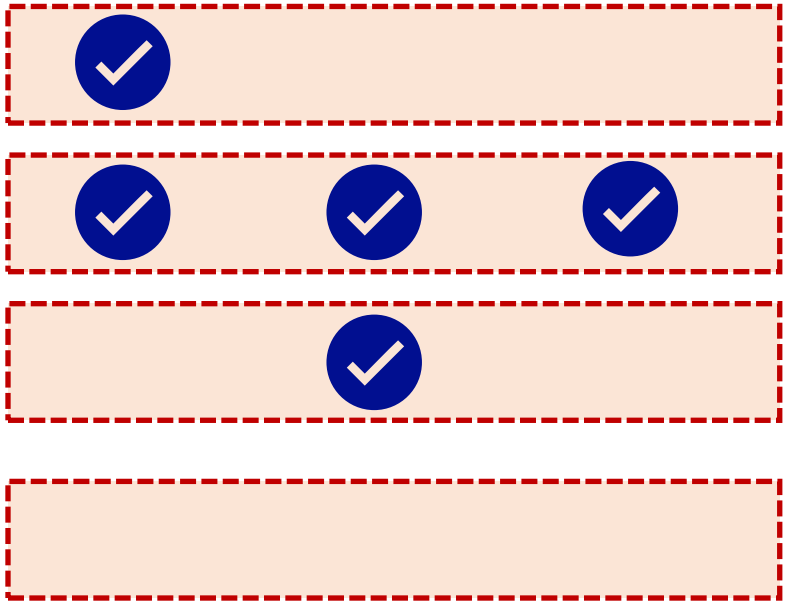
Which of the following belongs to $O(n^3)$ $\Omega(n^3)$ $\Theta(n^3)$

$$f_1(n) = 77n^2$$

$$f_2(n) = 6n^3 + n^2 \log n$$

$$f_3(n) = 11n^4$$

$$f_4(n) = \begin{cases} n^2 & n \text{ is even} \\ n^4 & n \text{ is odd} \end{cases}$$



Properties of Big-Ω and Big-Θ

$f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n))$

$f(n) \in \Theta(g(n)) \Leftrightarrow f(n) \in O(g(n))$ and $g(n) \in O(f(n))$

$f(n) \in \Theta(g(n)) \Leftrightarrow f(n) \in O(g(n))$ and $f(n) \in \Omega(g(n))$

Summary of asymptotic notations: Big-O, Big-Ω and Big-Θ

- $O(g(n))$ is a set of functions. For $f(n) \in O(g(n))$
  - intuitively, $f(n)$ is no worse than $g(n)$
  - formally, there exist c, n_0 such that $f(n) \leq c\ g(n)$ for all $n \geq n_0$

"$\leq$"

Upper bound

- $\Omega(g(n))$ is a set of functions. For $f(n) \in \Omega(g(n))$
  - intuitively, $f(n)$ is no better than $g(n)$
  - formally, there exist c, n_0 such that $f(n) \geq c\ g(n)$ for all $n \geq n_0$

"$\geq$"

Lower bound

- $\Theta(g(n))$ is a set of functions. For $f(n) \in \Theta(g(n))$
  - intuitively, $f(n)$ is equivalent to $g(n)$
  - formally, there exist c_0, c_1, n_0 such that $c_0 g(n) \leq f(n) \leq c_1\ g(n)$ for all $n \geq n_0$

"$=$"

Upper and lower bound

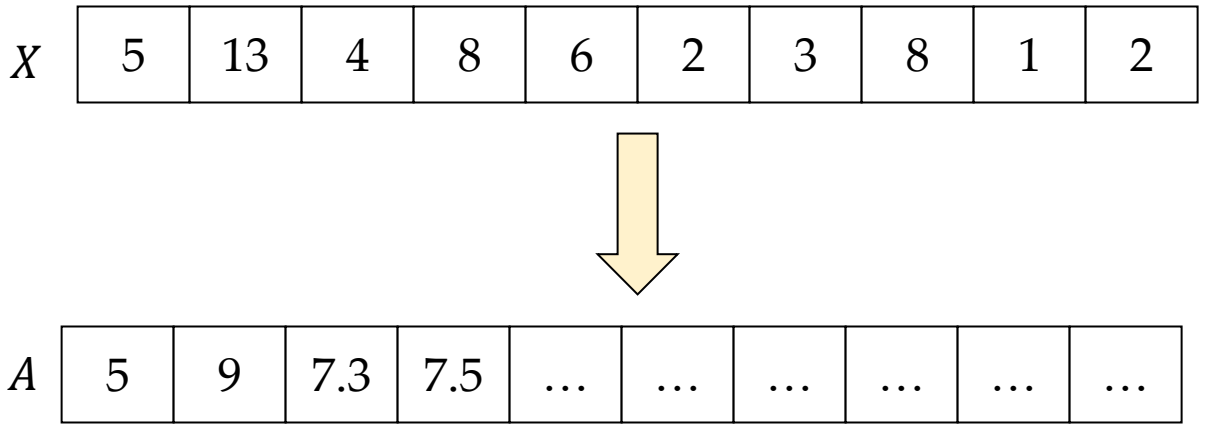
Analysis of algorithms: roadmap

- What is an algorithm?
- Evaluating running time of an algorithm
  - experimental study
  - operations counting
- Big-*O*
  - Big-*O* definition
  - properties of Big-*O*
  - asymptotic analysis
- Big-Ω and Big-Θ
- Comparing two algorithms
- Math reviews

Computing prefix averages

- Input: an array X of n numbers
- Output: an array A of prefix averages of X

$$A[i] = \frac{\sum_{j=0}^{i} X[j]}{i+1} = \frac{X[0]+X[1]+\cdots+X[i]}{i+1} \text{ for all } i = 0,1,\ldots,n-1$$



Algorithm 1: prefixAverages1

```
Algorithm prefixAverages1(X,n)
Input: array X of n numbers
Output: array A of prefix averages of X
0  A ← new array of n numbers
1  For i ← 0 to n − 1 do                    ▷ O(n) iterations over output array
2      s ← X[0]                             ▷ O(n) (O(1) ops for O(n) times)
3      For j ← 1 to i do                    ▷ O(1 + 2 + ··· + n)
4          s ← s + X[j]
5      A[i] ← s/(i + 1)                     ▷ O(n) (O(1) ops for O(n) times)
6  Return A
```

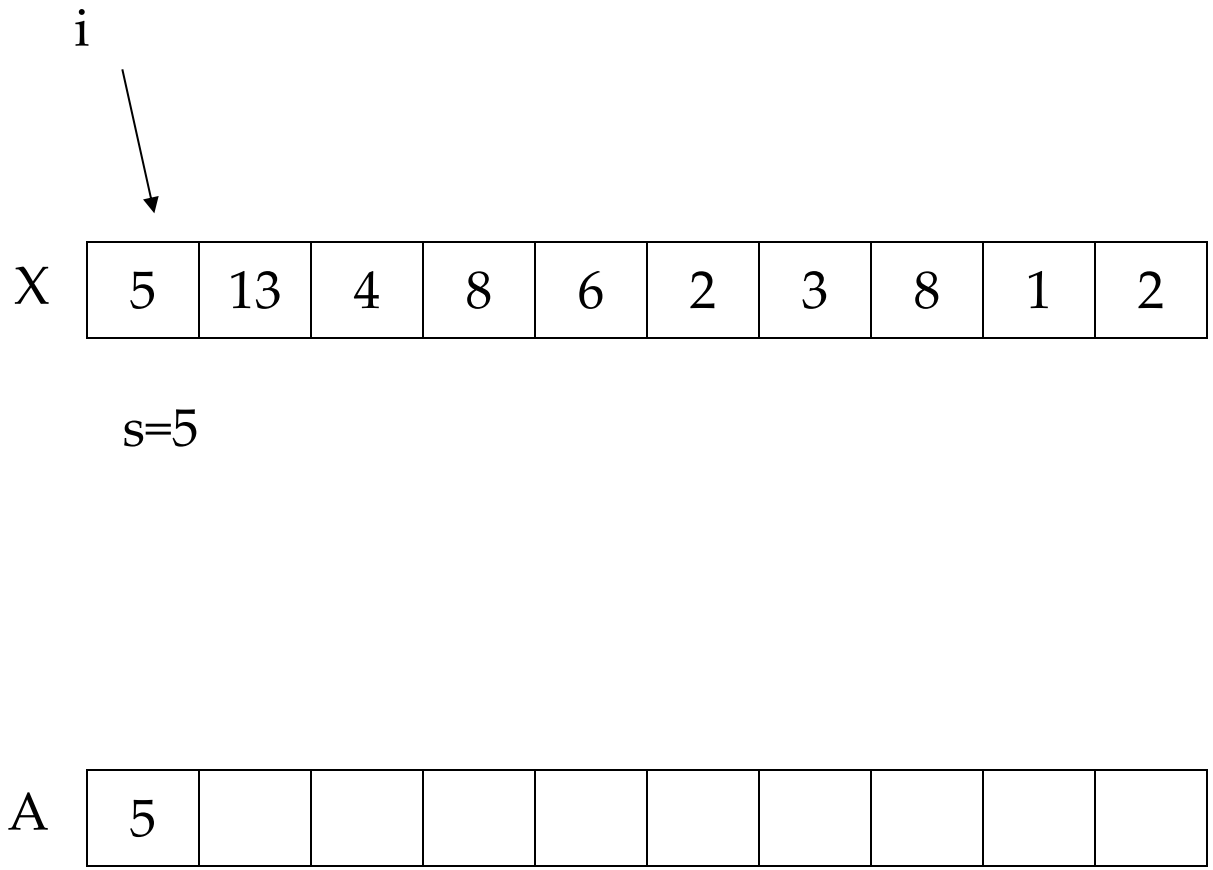
The RT of prefixAverages1 is $O(1 + 2 + \cdots + n) = O(n^2)$

Arithmetic progression!

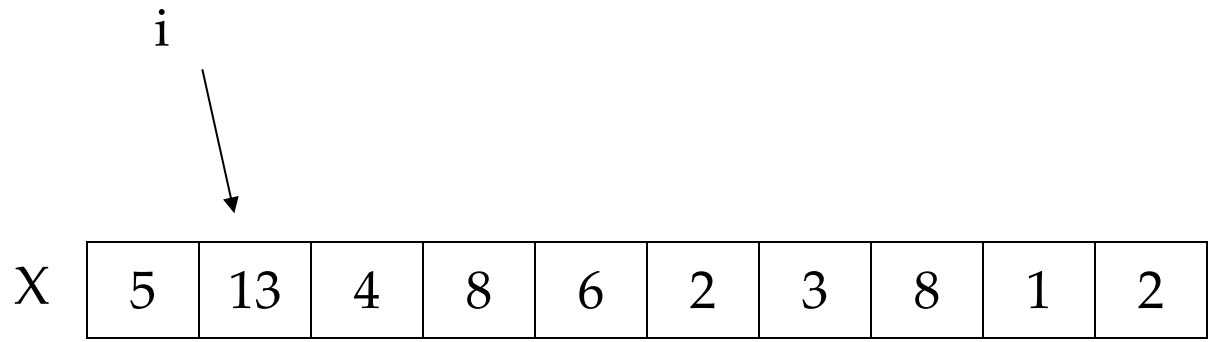
Algorithm 2: prefixAverages2

```
Algorithm prefixAverages2(X,n)
Input: array X of n numbers
Output: array A of prefix averages of X
0  A ← new array of n numbers
1  s ← 0
2  For i ← 0 to n − 1 do
3     s ← s + X[i]
4     A[i] ← s/(i + 1)
5  Return A
```

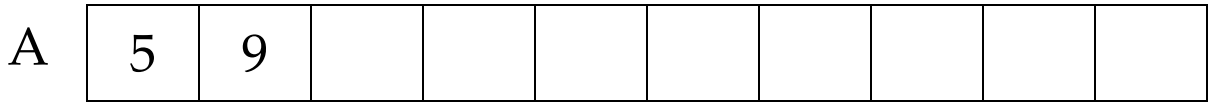
```
1   s ← 0
2   For i ← 0 to n − 1 do
3       s ← s + X[j]
4       A[i] ← s/(i + 1)
```



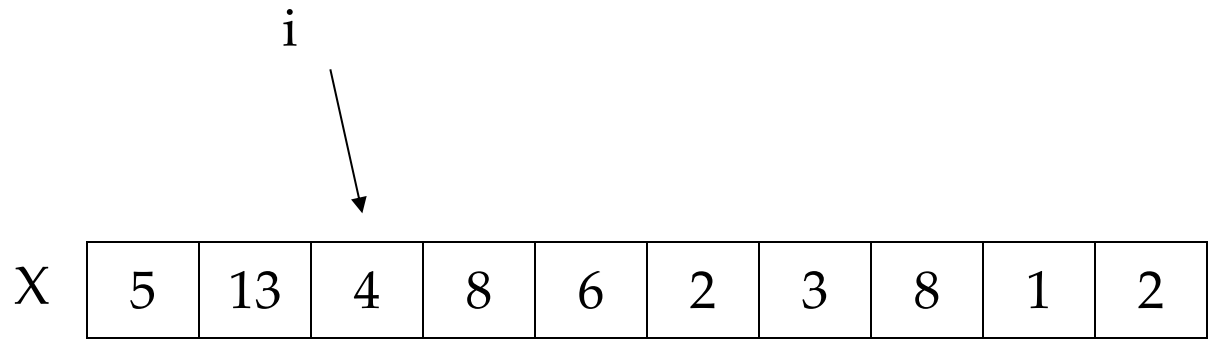
```
1   s ← 0
2   For i ← 0 to n − 1 do
3       s ← s + X[j]
4       A[i] ← s/(i + 1)
```



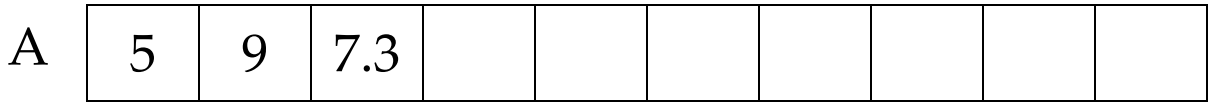
s=18



```
1   s ← 0
2   For i ← 0 to n − 1 do
3       s ← s + X[j]
4       A[i] ← s/(i + 1)
```



s=22



Comparing two algorithms

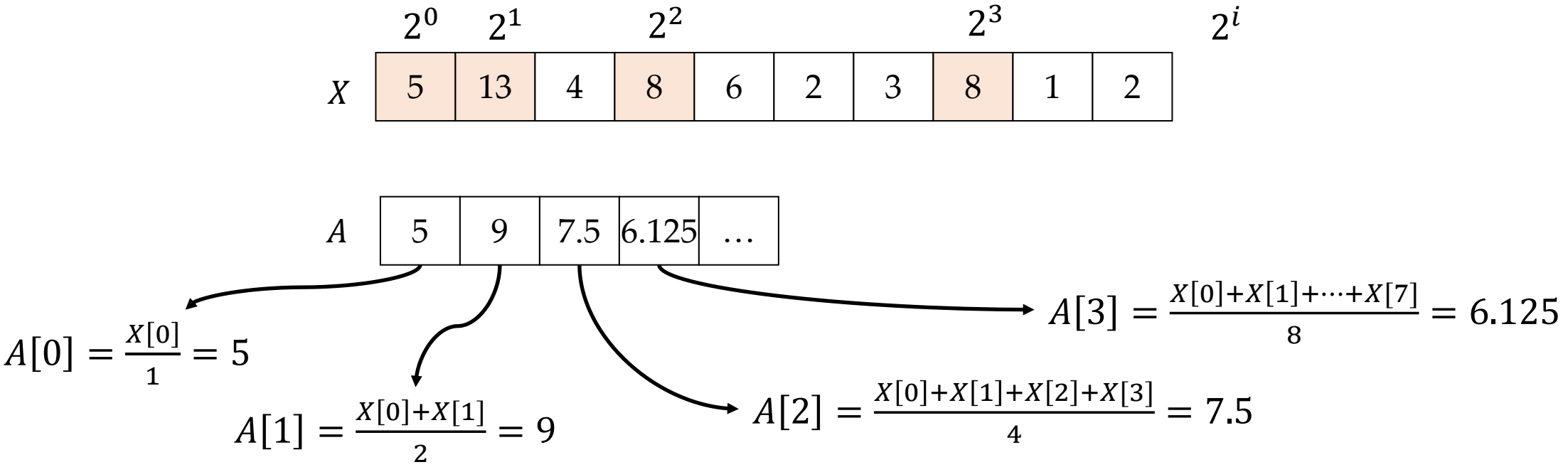
```
Algorithm prefixAverages2(X,n)
Input: array X of n integers
Output: array A of prefix averages of X
0  A ← new array of n numbers
1  s ← 0
2  For i ← 0 to n − 1 do                    ▷ O(n) iterations over input array
3    s ← s + X[i]                           ▷ O(n) (O(1) ops for O(n) times)
4    A[i] ← s/(i + 1)
5  Return A
```

- The RT of prefixAverages2 is $O(n)$
- prefixAverages2 is more efficient than prefixAverages1: $O(n^2)$ quadratic vs. $O(n)$ linear

Computing prefix averages at geometric time steps

- Input: an array X of n numbers
- Output: an array A of prefix average for X only at geometric time steps

$$A[i] = \frac{\sum_{j=0}^{2^i-1} X[j]}{2^i} = \frac{X[0]+X[1]+\cdots+X[2^i-1]}{2^i} \text{ for all } i = 0,1,\ldots,\log n$$



Algorithm 1: prefixAveragesGeo1

```
Algorithm prefixAveragesGeo1(X, n)
Input: array X of n numbers
Output: array A of prefix averages of X
0  A ← new array of log n + 1 numbers
1  For i ← 0 to log n do                          ▷ O(log n) iterations over output array
2      s ← X[0]                                   ▷ O(log n)
3      For j ← 1 to 2^i − 1 do                    ▷ O(1 + 2 + 2^2 ... + 2^(log n))
4          s ← s + X[j]
5      A[i] ← s/2^i                               ▷ O(log n)
6  Return A
```

The RT of prefixAveragesGeo1 is $O(1 + 2 + \cdots + 2^{\log n})$

Needs to be simplified

Geometric progression

$$f(k) = \sum_{i=0}^{k} 2^i = 1 + 2 + 4 \ldots + 2^k = 2^{k+1} - 1$$

Proof (Algebra):

$$f(k) = 1 + 2^1 + 2^2 + \cdots + 2^{k-1} + 2^k$$

$$2 \cdot f(k) = 2^1 + 2^2 + 2^3 + \cdots + 2^k + 2^{k+1}$$

$$2f(k) - f(k) = 2^1 + 2^2 + 2^3 + \cdots + 2^k + 2^{k+1}$$
$$-1 - 2^1 - 2^2 - \cdots - 2^{k-1} - 2^k$$

$$\Rightarrow f(k) = 2^{k+1} - 1$$

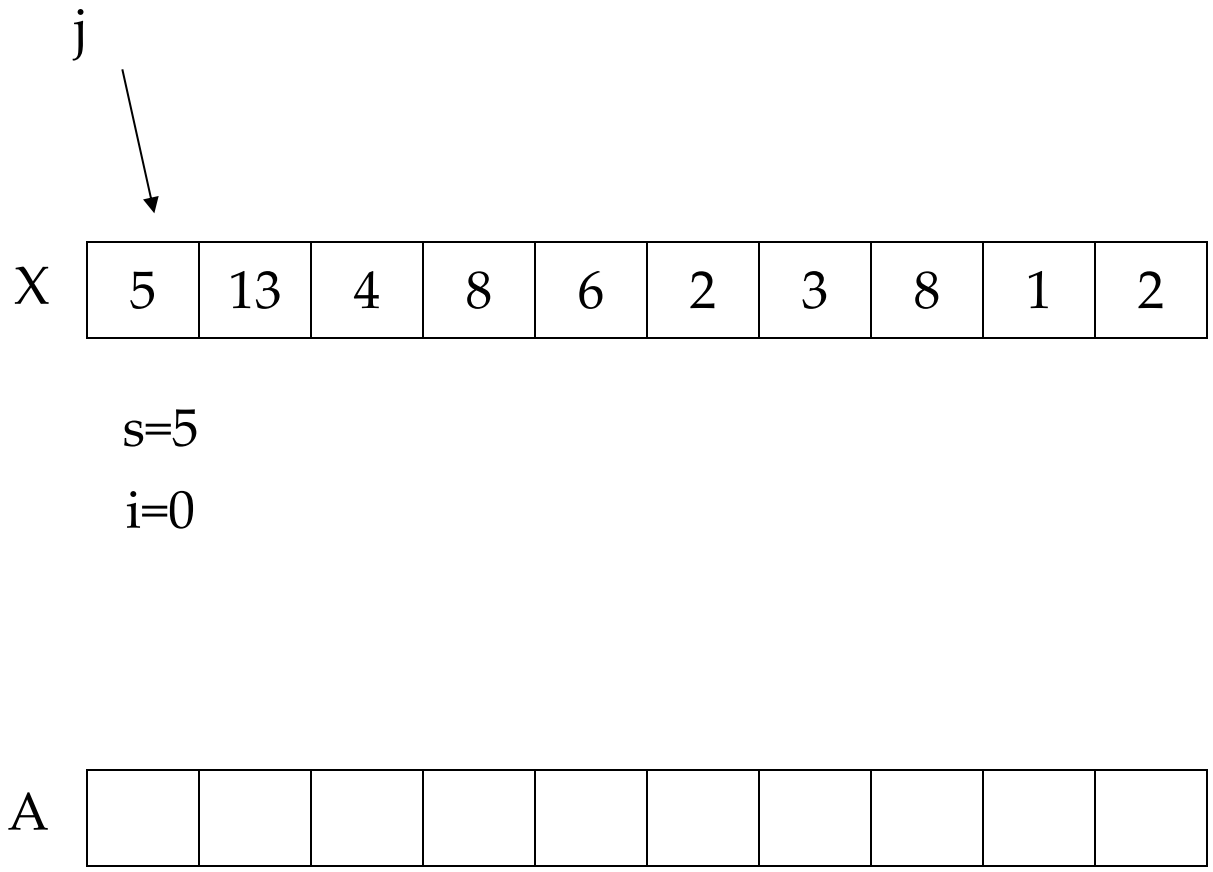
Algorithm 1: prefixAveragesGeo1

```
Algorithm prefixAveragesGeo1(X,n)
Input: array X of n numbers
Output: array A of prefix averages of X
0  A ← new array of log n + 1 numbers
1  For i ← 0 to log n do                          ▷ O(log n) iterations over output array
2      s ← X[0]                                   ▷ O(log n)
3      For j ← 1 to 2^i − 1 do                    ▷ O(1 + 2 + 2^2 ... + 2^(log n))
4          s ← s + X[j]
5      A[i] ← s/2^i                               ▷ O(log n)
6  Return A
```

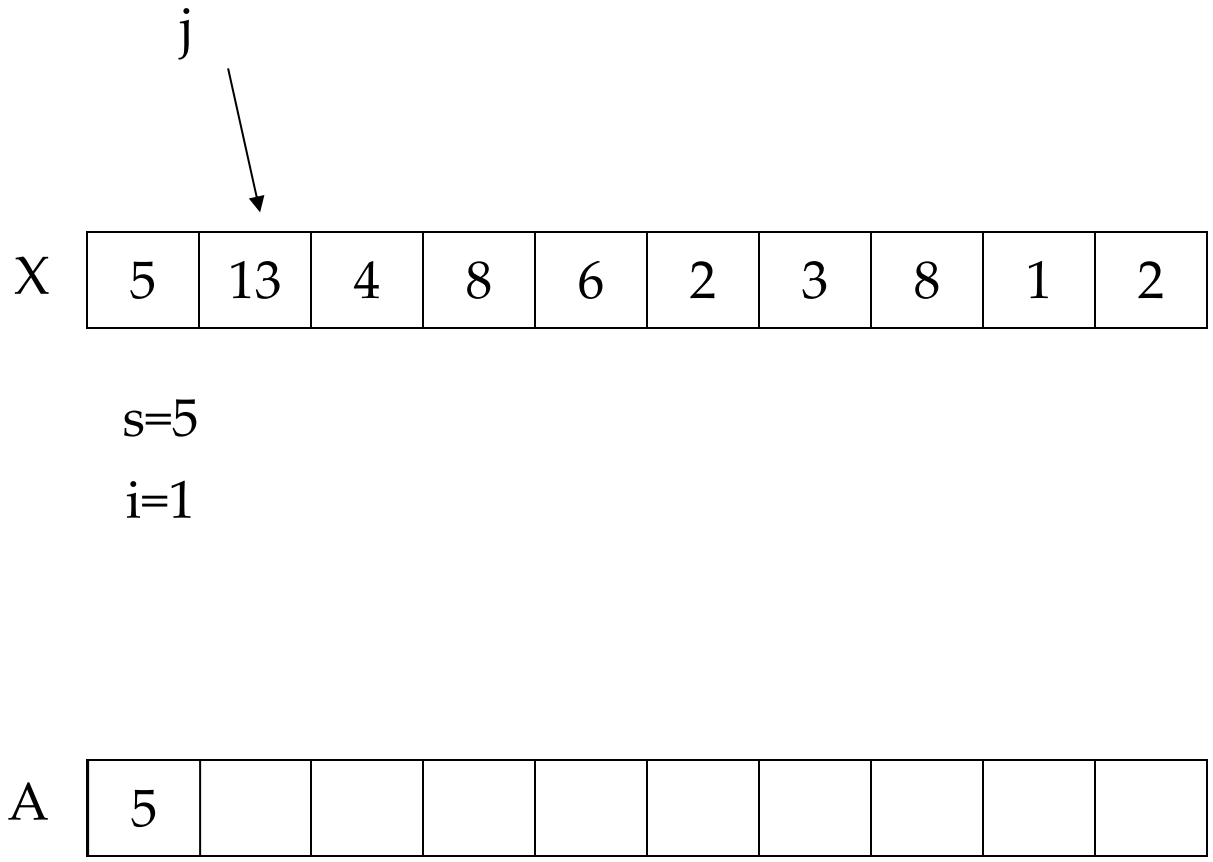
The RT of prefixAveragesGeo1 is $O(1 + 2 + \cdots + 2^{\log n}) = O(2^{\log n+1} - 1) = O(n)$

Algorithm 2: prefixAveragesGeo2

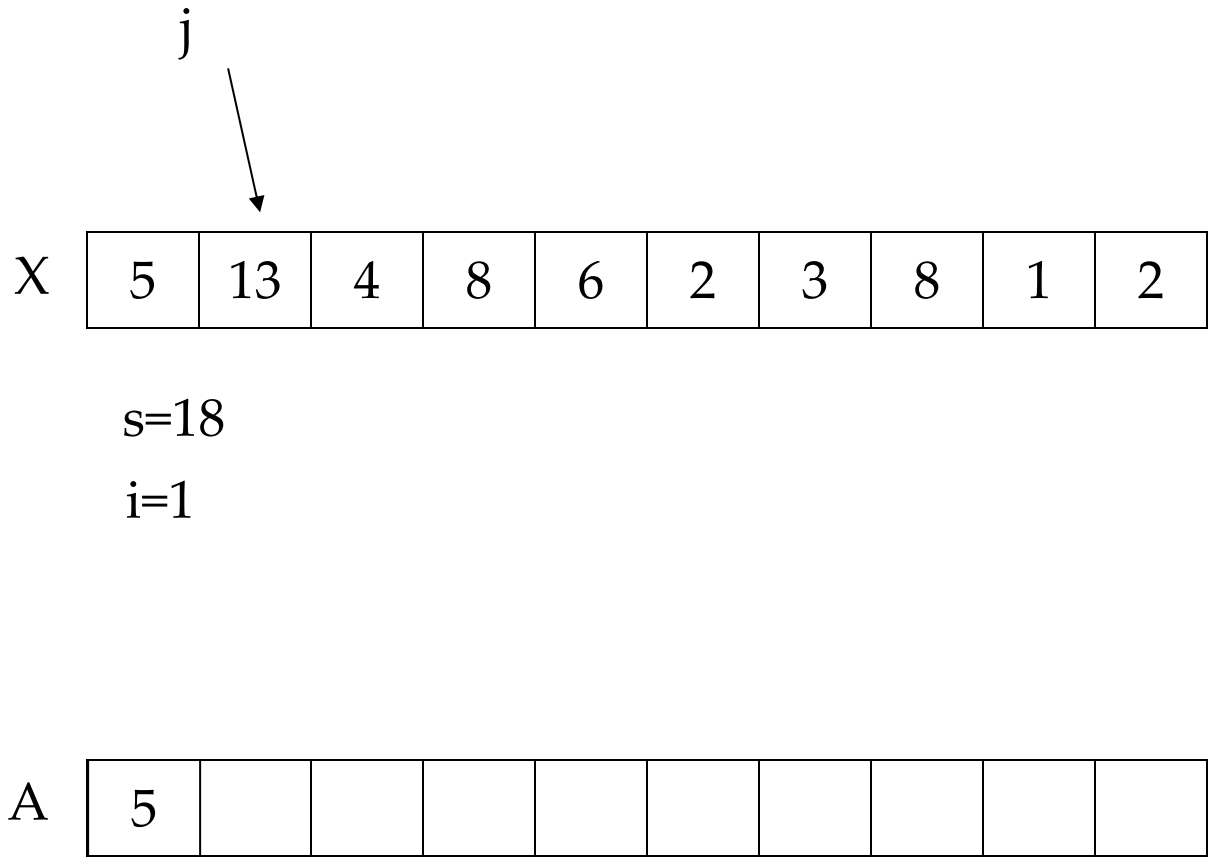
```
   Algorithm prefixAveragesGeo2(X, n)
   Input: array X of n integers
   Output: array A of prefix averages of X
0  A ← new array of log n + 1 numbers
1  s ← 0
2  i ← 0
3  For j ← 0 to n − 1 do
4     s ← s + X[j]
5     If j = 2^i − 1 then
6        A[i] ← s/(j + 1)
7        i ← i + 1
8  Return A
```



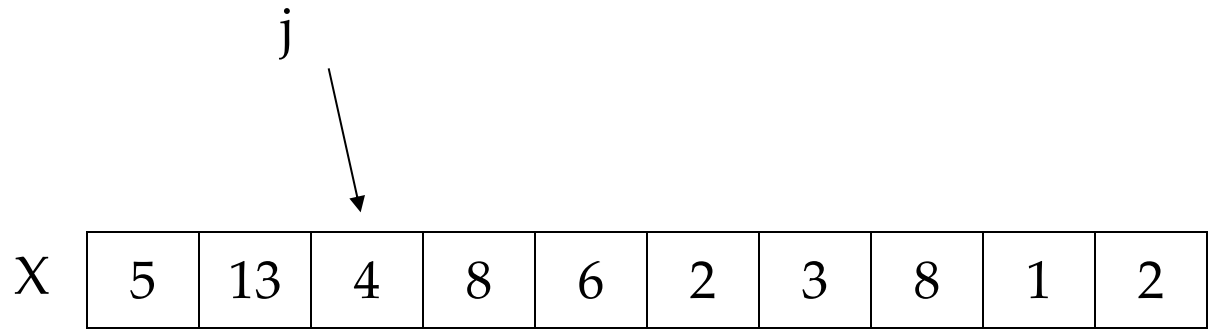

```
1  s ← 0
2  i ← 0
3  For j ← 0 to n − 1 do
4      s ← s + X[j]
5      If j = 2^i − 1 then
6          A[i] ← s/(j + 1)
7          i ← i + 1
```




```
1   s ← 0
2   j ← 0
3   For j ← 0 to n − 1 do
4       s ← s + X[j]
5       If j = 2^i − 1 then
6           A[i] ← s/(j + 1)
7           i ← i + 1
```

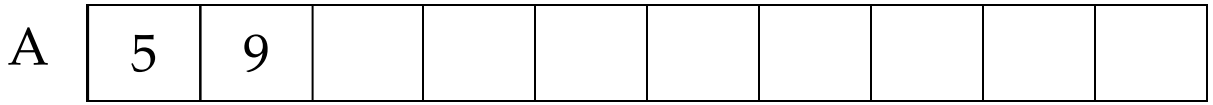



```
1  s ← 0
2  i ← 0
3  For j ← 0 to n − 1 do
4      s ← s + X[j]
5      If j = 2^i − 1 then
6          A[i] ← s/(j + 1)
7          i ← i + 1
```

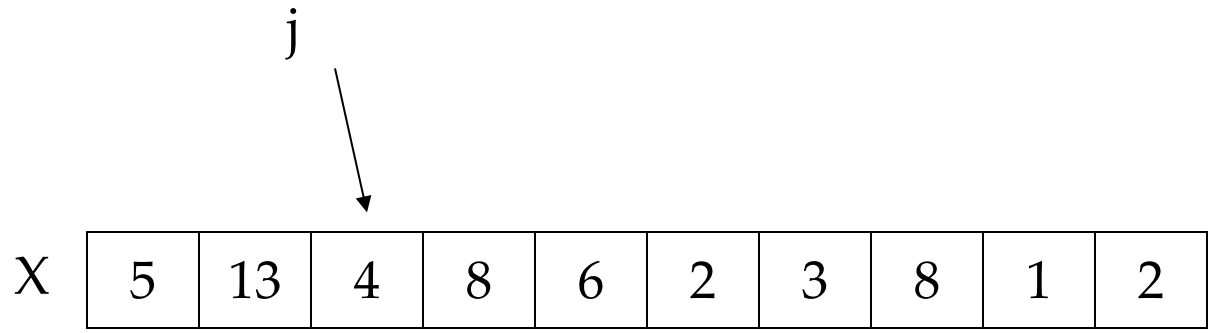


s=18

i=2

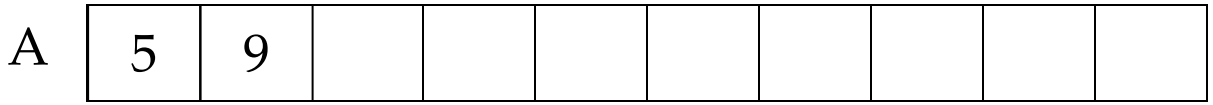



```
1  s ← 0
2  j ← 0
3  For j ← 0 to n − 1 do
4      s ← s + X[j]
5      If j = 2^i − 1 then
6          A[i] ← s/(j + 1)
7          i ← i + 1
```

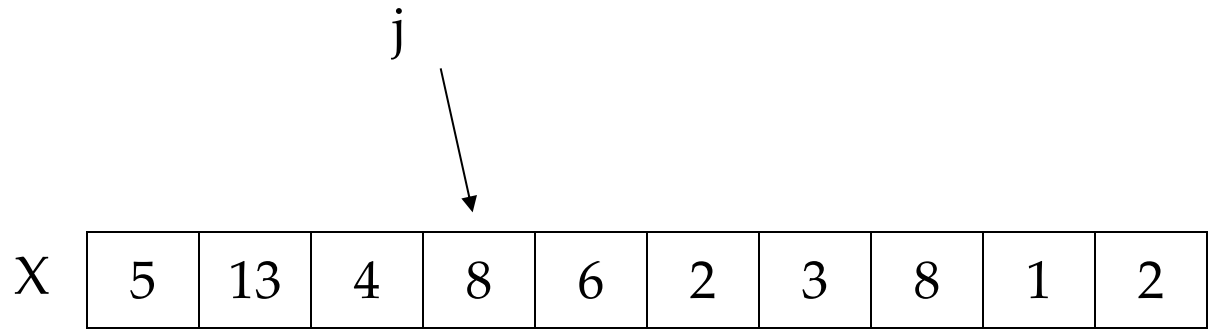


s=22

i=2

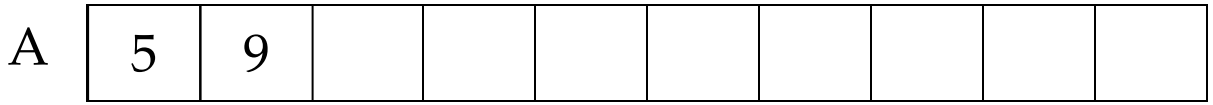



```
1  s ← 0
2  i ← 0
3  For j ← 0 to n − 1 do
4      s ← s + X[j]
5      If j = 2^i − 1 then
6          A[i] ← s/(j + 1)
7          i ← i + 1
```

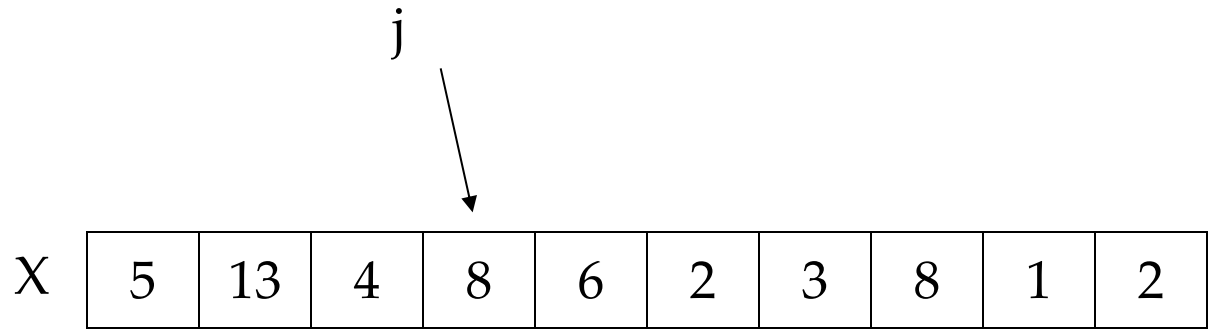


s=22

i=2

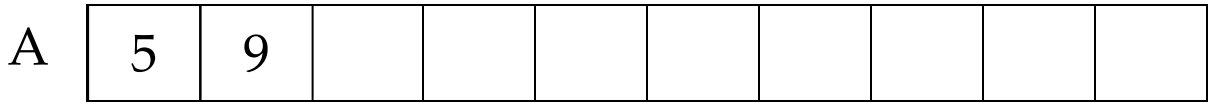



```
1  s ← 0
2  i ← 0
3  For j ← 0 to n − 1 do
4      s ← s + X[j]
5      If j = 2^i − 1 then
6          A[i] ← s/(j + 1)
7          i ← i + 1
```

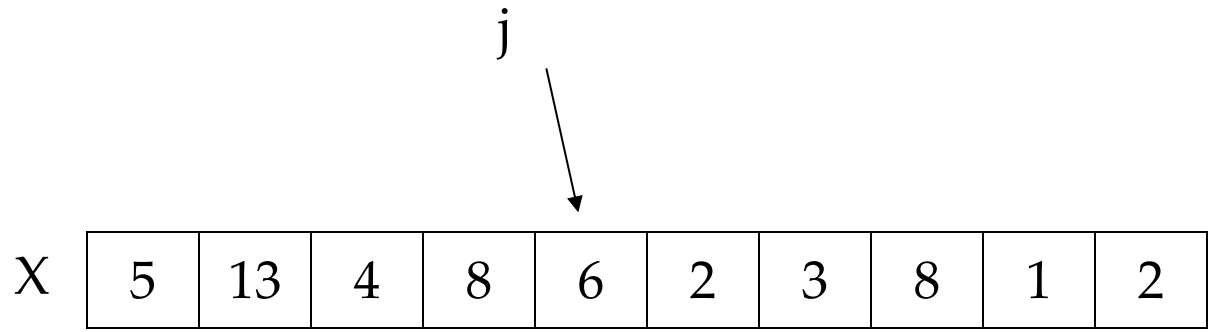


s=30

i=2




```
1  s ← 0
2  i ← 0
3  For j ← 0 to n − 1 do
4      s ← s + X[j]
5      If j = 2^i − 1 then
6          A[i] ← s/(j + 1)
7          i ← i + 1
```



s=30

i=3

A	5	9	7.5							

```
1  s ← 0
2  j ← 0
3  For j ← 0 to n − 1 do
4      s ← s + X[j]
5      If j = 2^i − 1 then
6          A[i] ← s/(j + 1)
7          i ← i + 1
```

Algorithm 2: prefixAveragesGeo2

```
Algorithm prefixAveragesGeo2(X,n)
Input: array X of n integers
Output: array A of prefix averages of X
0  A ← new array of log n + 1 numbers
1  s ← 0
2  i ← 0
3  For j ← 0 to n − 1 do                    ▷ O(n) iterations over input array
4      s ← s + X[j]                         ▷ O(n)
5      If j = 2^i − 1 then                  ▷ O(n)
6          A[i] ← s/(j + 1)                 ▷ O(log n)
7          i ← i + 1
8  Return A
```

- The RT of prefixAveragesGeo2 is $O(n)$
- prefixAveragesGeo2 and prefixAveragesGeo1 are asymptotically equivalent

Analysis of algorithms: roadmap

- What is an algorithm?
- Evaluating running time of an algorithm
  - experimental study
  - operations counting
- Big-*O*
  - Big-*O* definition
  - properties of Big-*O*
  - asymptotic analysis
- Big-Ω and Big-Θ
- Comparing two algorithms
- Math reviews

Math you need to know

- Properties of powers:

  $a^{(b+c)} = a^b a^c$

  $a^{bc} = (a^b)^c$

  $a^b / a^c = a^{(b-c)}$

  $b = a^{\log_a b}$

  $b^c = a^{c*\log_a b}$

- Properties of logarithms:

  $\log_b(xy) = \log_b x + \log_b y$

  $\log_b (x/y) = \log_b x - \log_b y$

  $\log_b x^a = a\log_b x$

  $\log_b a = \log_x a / \log_x b$

Math you need to know

- Floor: $\lfloor x \rfloor$ is the largest integer $\leq x$
- Ceiling: $\lceil x \rceil$ is the smallest integer $\geq x$

Math you need to know

- Arithmetic progression

$$f(n) = \sum_{i=0}^{n} i\,d = 0 + d + 2d + \cdots + nd = d\frac{n(n+1)}{2} \in \Theta(n^2)$$

- Geometric progression

$$f(n) = \sum_{i=0}^{n} r^i = 1 + r + r^2 + \cdots + r^n = \frac{r^{n+1}-1}{r-1} \in \Theta(r^n)$$

Next Topic

Stacks, Queues, Deques

Lists